

Efficient Fault Tolerance for Stateful Serverless Computing with Asymmetric Logging

SHENG QI, Peking University, China

HAOYU FENG, Peking University, China

XUANZHE LIU, Peking University, China

XIN JIN, Peking University, China

Serverless computing separates function execution from state management. Simple retry-based fault tolerance might corrupt the shared state with duplicate updates. Existing solutions employ log-based fault tolerance to achieve exactly-once semantics, where every single read or write to the external state is associated with a log for deterministic replay. However, logging is not a free lunch, which introduces considerable overhead to stateful serverless applications.

We present Halfmoon, a serverless runtime system for fault-tolerant stateful serverless computing. Our key insight is that it is unnecessary to symmetrically log *both* reads and writes. Instead, it suffices to log *either* reads *or* writes, i.e., asymmetrically. We design two logging protocols that enforce exactly-once semantics while providing log-free reads and writes, which are suitable for read- and write-intensive workloads, respectively. We theoretically prove that the two protocols are *log-optimal*, i.e., no other protocols can achieve lower logging overhead than our protocols. We provide a criterion for choosing the right protocol for a given workload, and a pauseless switching mechanism to switch protocols for dynamic workloads. We implement a prototype of Halfmoon. Experiments show that Halfmoon achieves 20%–40% lower latency and 1.5–4.0× lower logging overhead than the state-of-the-art solution Boki.

1 INTRODUCTION

Recent years have seen an increasing popularity of serverless computing [17, 34, 35, 38, 39, 55, 97, 100, 109, 123] in building cloud applications [53, 83, 94, 102, 115, 121]. It features the Function-as-a-Service (FaaS) paradigm [95, 110, 126], where developers break down an application into a set of functions and their dependencies, and the cloud automates the deployment. FaaS enjoys elastic scaling and pay-per-use billing, which dramatically reduces resource management overhead.

This work is an extended version of the paper that appeared in SOSP 2023 [91]. The additional content includes:

- (1) A full proof of idempotence for the Halfmoon-read protocol (§4.1.2)
- (2) A full proof of idempotence for the Halfmoon-write protocol (§4.2.2)
- (3) A full proof of consistency for the Halfmoon-read and Halfmoon-write protocol (§4.4)
- (4) A full proof of idempotence and consistency for the hybrid protocol that combines Halfmoon-read and Halfmoon-write at object granularity (§4.5)
- (5) A complete description of the switching procedure between the Halfmoon-read and Halfmoon-write protocols (§4.8), including:
 - (a) a locking primitive to atomically commit single- and multi-version writes, and the implementation of the transitional read and write operations based on the locking primitive (§4.8.2)
 - (b) a discussion on the idempotence and consistency of the switching procedure (§4.8.3)
 - (c) a mechanism to ensure that the external state is up-to-date after the switching (§4.8.4).
- (6) An extension to the Halfmoon-write protocol that enforces strict ordering of consecutive writes (§6.1).
- (7) An extension to the Halfmoon-read and Halfmoon-write protocols using a partially ordered log (§6.2).
- (8) An extended evaluation of Halfmoon, including:
 - (a) An evaluation of Halfmoon’s read and write primitives under various request rates (§7.1).
 - (b) An evaluation of Halfmoon’s end-to-end performance in the presence of function failures and retries (§7.5).
 - (c) An evaluation of Halfmoon’s end-to-end performance when overlapping logging with program execution (§7.6).
 - (d) An evaluation of the extended Halfmoon-write protocol that enforces strict ordering of consecutive writes (§7.7).

Authors’ Contact Information: Sheng Qi, Peking University, China; Haoyu Feng, Peking University, China; Xuanzhe Liu, Peking University, China; Xin Jin, Peking University, China.

Table 1. Latency of log, read and write operations in Boki.

	Log	Read	Write
median	1.18ms	1.88ms	2.47ms
99%-tile	1.91ms	4.60ms	5.86ms

Serverless platforms enable autoscaling by disaggregating compute and storage [44, 68]. Function-local state is not guaranteed to persist across invocations due to load balancing and elastic scaling of resources. To share state across multiple functions, applications typically rely on external storage for state management [11].

Extracting the state from stateful serverless functions (SSFs) [52, 119] brings challenges to application-level consistency in the presence of failures. While SSFs are decomposed into stateless functions and the external state, achieving fault tolerance of SSFs is not as simple as achieving fault tolerance for each component individually. Specifically, the fault tolerance of stateless functions can be achieved by retrying crashed functions, and that of the external state can be achieved by using a fault-tolerant external storage service. However, naively combining the two introduces anomalies upon failures. Consider an SSF that writes to the external state and then crashes. Retrying this function would duplicate the write that has already been applied.

Serverless runtimes should avoid such anomalies with exactly-once semantics [96, 119]. That is, no matter how many times an SSF crashes and gets re-executed, the effect on the external state should be equivalent to that produced by running the SSF exactly once, without crashing.

Log-based fault tolerance is a common approach to realizing exactly-once semantics [26, 96]. The idea is to enhance the retry-based at-least-once semantics with idempotence, i.e., at-most-once semantics. To achieve this, existing solutions associate every read or write to the external state with a log record. During re-execution, the SSF replays the log, recovering read results and skipping completed writes. Beldi proposes to atomically perform writing and logging in the external storage [119]. The state-of-the-art solution Boki decouples the log to a more efficient logging layer [52].

However, logging is not a free lunch. Beldi reports that reads and writes with logging are 2-4× more expensive than their raw counterparts. Even for Boki’s optimized implementation, logging still accounts for 30%-50% overhead compared to raw operations (§2). Thus the primary goal of this paper is to provide idempotence while minimizing logging overhead.

We present Halfmoon, a serverless runtime system for fault-tolerant SSFs. Halfmoon provides two logging protocols that enforce exactly-once semantics while providing log-free reads and writes, respectively. Our intuition is that there is no need to symmetrically log *both* reads and writes. Instead, it suffices to log *either* reads *or* writes, i.e., asymmetrically. Consider the stream of events that take place in the system. Our key insight is that reads and writes are *parameterized* by their timestamps in the stream, and idempotence boils down to the stability of timestamps [50]. An idempotent write should always be applied at the same point in the stream, i.e., at the same timestamp. Similarly, an idempotent read should always seek backward from its timestamp and observe the latest preceding write in the stream.

The key challenge of Halfmoon’s asymmetric design is persisting events without logging. While a logged event (and its timestamp) is persistent on its own, an unlogged event must be recovered from other logged events after failure. The problem is that the assignment of timestamps is inherently non-deterministic. Halfmoon addresses this problem by leveraging the fact that many serverless applications do not require real-time consistency [98, 99, 111]. Instead of assigning the non-recoverable real time to log-free reads or writes, we generate their timestamps based on those of previous logged operations, in a *deterministic* and *recoverable* fashion. This allows us to eliminate the logging overhead for one type of operation, and rely on the log records of the other type to achieve both persistence and idempotence. We discuss Halfmoon’s consistency guarantees in §4.4.

We prove that the two logging protocols are *log-optimal* for exactly-once semantics, i.e., no other protocols can achieve exactly-once semantics with lower worst-case logging overhead than our protocols. Intuitively, the two protocols are suitable for read- and write-intensive workloads, respectively. We provide theoretical and empirical analysis of Halfmoon under different read/write intensities, and propose a criterion for choosing the right protocol for a given workload.

Halfmoon also supports dynamic workloads that change their read/write intensity over time. We design a switching procedure that allows the runtime to change between Halfmoon’s log-free read protocol and log-free write protocol. The switching is *pauseless*, i.e., the system remains operational and fault-tolerant during this process.

In summary, we make the following contributions.

- We design two logging protocols that enable exactly-once log-free reads and writes for SSFs, respectively.
- We theoretically prove that our protocols are log-optimal. Therefore, Halfmoon pushes the overhead of log-based fault tolerance to its lower bound.
- We provide a criterion for choosing the right protocol for a given workload, and a pauseless switching procedure to switch protocols for dynamic workloads.
- We implement a prototype of Halfmoon. Our experiments show that Halfmoon achieves 20%–40% lower latency and 1.5–4.0× lower logging overhead than the state-of-the-art solution Boki.

2 MOTIVATION

Problem: logging overhead. To achieve exactly-once semantics for SSFs, existing solutions [52, 119] require logging for *both* reads and writes to the external state. Note that reads should be idempotent because writes and the branching of SSF logic may arbitrarily depend on read results. We refer to these fault-tolerant logging protocols as *symmetric*. Boki [52] is the state-of-the-art symmetric approach that specifically optimizes its logging implementation. Nevertheless, logging still incurs substantial overhead. We benchmark Boki’s logging latency under the setup of §7, using Amazon DynamoDB as the external storage [36]. Table 1 shows that logging accounts for 63% (48%) overhead at the median and 42% (33%) at the 99%-tile for reads (writes). This motivates us to design minimally fault-tolerant protocols for SSFs that perform logging only if necessary.

Opportunity: parameterizing reads and writes. The event stream is the key enabler of Halfmoon’s design. The ordering of events in the stream can be obtained through sequencers [19, 31, 52, 70], synchronized clocks [28, 59, 73], or state machine replication (SMR) [14, 29, 37, 61]. Without loss of generality, we assume for now that each operation is associated with a timestamp and the event stream follows the timestamp order. We further discuss the availability of the event stream in §8.

Our key insight is that reads and writes are *parameterized* by their timestamps in the event stream. To achieve idempotence, a read should consistently seek backward from the same timestamp. Figure 1 shows a real-time interleaving of SSFs and the event timestamps. The first time F2 executes read at t_1 , it sees the latest write from F1 at t_0 . During re-execution, instead of seeing F3’s write at t_3 , it should also seek backward from t_1 and recover the previous result, i.e., the value written at t_0 . Similarly, for a write to be idempotent, it should always take effect at the same point in the stream regardless of re-execution. As shown in Figure 1, when F2 recovers from failure, it should avoid overwriting F3’s write because F2’s write has been parameterized at t_2 , i.e., it should take effect before F3’s write.

By parameterizing reads and writes, idempotence boils down to the stability of read and write timestamps [50]. This motivates us to rethink the necessity of existing logging protocols. Our intuition is that there is no need to symmetrically

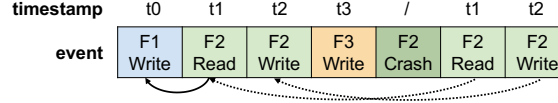


Fig. 1. Parameterizing reads and writes with timestamps.

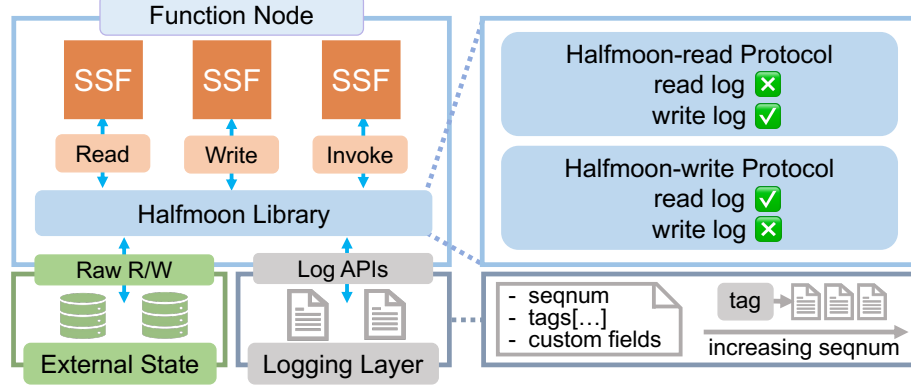


Fig. 2. Halfmoon overview.

log *both* reads and writes. Instead, by leveraging the inherent dependencies between events, it suffices to log *either* reads *or* writes, i.e., asymmetrically.

Challenge: persistence without logging. Parameterizing reads and writes does not provide fault tolerance on its own. The critical part is to persist the event stream in the first place, such that timestamps and the ordering of events remain stable across failures. This property is implicitly satisfied by symmetric logging protocols, as logged events are persistent by themselves. However, Halfmoon’s design goal of being log-free presents a new challenge to persisting the event stream. For an unlogged event, the only way to persist it is to make sure that it can be recovered from other logged events. Consequently, the dependencies of this event must be deterministic. By requiring that SSFs be deterministic [119], the input parameters to reads or writes can be stably inferred from function context. However, the assignment of event timestamps is inherently non-deterministic, which must be logged to rule out the uncertainty.

At first glance, this seems to be contradictory to our design goal. To address this problem, Halfmoon leverages the fact that many serverless applications function well under less stringent guarantees than real-time consistency, i.e., linearizability. For example, Cloudburst [99] provides *repeatable reads* or *causal consistency* for SSFs; AFT [98] provides *read atomicity*. Therefore, the read/write timestamps do not have to reflect the real time, which is non-deterministic and unrecoverable without logging. Instead, we can deterministically generate *logical* timestamps for log-free operations based on the SSF context, similar to the way we infer the input parameters for reads or writes. In other words, it is possible to infer all events from a skeleton of the event stream. Only the skeleton needs to be persisted, thereby saving the logging overhead for the rest.

3 HALFMOON OVERVIEW

Figure 2 shows the overall architecture of Halfmoon. SSFs interact with the external state through Halfmoon’s client library. The library exposes similar APIs as existing solutions [52, 119], including data operations such as read/write,

```

# Return the sequence number of the log record
def logAppend(tags, record) -> seqnum
# Read the previous or next log record
# whose seqnum <= `max_seqnum` or >= `min_seqnum`
def logReadPrev(tag, max_seqnum) -> LogRecord
def logReadNext(tag, min_seqnum) -> LogRecord
# Delete log records up to `seqnum`
def logTrim(tag, seqnum)
# Conditional log append (Section 5.1)
def logCondAppend(tags, record, condTag, condPos)
    -> (seqnum, error)

```

Fig. 3. The log APIs in Halfmoon.

and control flow operations such as `invoke` to enable stateful workflows. The APIs have the same signature as their raw counterparts, but automatically performs logging behind the scene to ensure idempotence. To achieve exactly-once semantics, Halfmoon relies on the serverless runtime to detect and re-execute crashed SSFs. This feature is widely supported among existing platforms [1, 7, 96, 119].

Halfmoon applies log-based fault tolerance with novel log-optimal protocols. The Halfmoon-read protocol is log-free on reads, and the Halfmoon-write protocol is log-free on writes. In line with the state-of-the-art solution Boki [52], Halfmoon decouples the log from the external state into a separate logging layer. The logging layer implements the shared log abstraction [20, 31, 52], which enforces a global total order of log records and serves as the event stream of Halfmoon. We note that Halfmoon is not tied to Boki’s logging layer (§8). Our prototype uses Boki because it specifically optimizes logging for stateful serverless computing.

Figure 3 lists Halfmoon’s log APIs. `logAppend` appends to the log and assigns a globally monotonically increasing sequence number (`seqnum`) to the log record. Each log record has a number of `tags` as specified in `logAppend`. The main log is logically divided into sub-streams where log records have a common `tag`, and a record may appear in several sub-streams. Because the order of records is determined by their `seqnums`, the order within each sub-stream is consistent with that of the main log. Sub-streams reduce log replay overhead by enabling selective reads, a common approach in shared log systems [21, 52, 107]. `logReadPrev` (`logReadNext`) seeks backward (forward) on a sub-stream specified by the `tag` parameter. `logTrim` garbage collects a sub-stream. Besides the APIs of prior work [52], we introduce `logCondAppend` to resolve conflicts between concurrent SSF instances (§5). Because all log APIs target specific sub-streams, we abbreviate sub-streams as streams for the rest of this paper.

4 HALFMOON DESIGN

This section presents the design of the Halfmoon-read and the Halfmoon-write protocol. We start by clarifying the concepts and assumptions.

Race conditions. Let *instances* denote executions of an SSF corresponding to a *single* function invocation. There can be multiple instances because an instance might crash and the serverless runtime would launch a new one to serve the SSF invocation. Without loss of generality, instances should be assigned a common identifier (`tag`) so that they may refer to the same log stream containing the SSF’s execution history. We use *instanceID* to denote this common identifier. The lifetime of instances may be *disjoint*, where a second instance starts after the first one has crashed; or *overlapping*, where the first one is assumed to have crashed by the runtime, but is still actually alive. This could happen

when the first instance fails to make progress in time, due to network errors or unexpected slow execution. We use *peers* specifically to denote such *concurrent* instances for an SSF invocation.

Consequently, there are two race conditions against the exactly-once semantics. First, an instance may race with another disjoint instance, at the risk of repeating a completed step. Second, an instance may also race with its peers, both attempting to execute the same step. For the sake of presentation, this section focuses on addressing the first race condition. We extend the protocols to handle the second one in §5. For now, we assume that the logging layer resolves conflicts between peers such that **logAppend** is idempotent; the formal proof is deferred to §5.

Time-related concepts. A call to the **logAppend** API returns a monotonically increasing *seqnum* from the logging layer (Figure 3). Within each SSF, we use a variable *cursorTS* to record the function-local seqnum that advances after each logging operation. As per §2, *timestamp* is a general concept that parameterizes the position of the associated event in the event stream. Note that depending on the logging protocol, timestamps can be based on seqnums (i.e., logical) or the real time.

Transactions. In line with previous works, we assume that SSFs are non-transactional by default [52]; to execute several steps atomically, SSFs should explicitly use transactions. Halfmoon can reuse existing transactional APIs, and focuses on optimizing the logging overhead for normal operations.

4.1 Halfmoon-Read: the Log-Free Read Protocol

Overview. Under Halfmoon-Read, all reads are log-free and only writes perform logging. The persistent part of the event stream consists only of writes. Writes achieve idempotence by checking the write logs in advance. For reads, we take two steps to ensure idempotence. First, we assign timestamps to reads in a deterministic manner. Second, we use the timestamps to map reads to existing writes in the write log. In other words, reads are inserted into the event stream given their timestamps, and are idempotent because their positions are deterministic.

In step one, we must infer the read timestamps from the SSF context. Our solution is to use *cursorTS*, i.e., the seqnum of the latest logged operation in the SSF; then the fault tolerance of the logging layer guarantees that read timestamps are deterministic.

In step two, we must ensure that writes are traceable. Our solution is to use multi-versioning to manage the external state. Each write creates a new version of the object and registers the version number in the logging layer; a read locates a specific object version by querying the write log (**logReadPrev**). Note that the version numbers are unordered by themselves; the write log defines the order. Therefore the external storage only needs to support plain key-value APIs; the version numbers serve as pointers to the actual object. Because the position of a write in the event stream is determined by the seqnum of the associated write log record, the write timestamp is set to that seqnum accordingly.

Example. Figure 4 shows an example of the Halfmoon-read protocol. F1’s initial *cursorTS* is t_0 . When reading object X, it does not see F2’s write at t_1 because the read uses an earlier timestamp. However, when reading object Y, it sees F2’s write at t_2 because it has advanced its *cursorTS* to t_3 after its previous write. The effective order of events follows the order of the seqnum-based logical timestamps. We show in §4.4 that Halfmoon-read provides *sequential consistency* [64].

4.1.1 The Halfmoon-Read Primitives.

Init. Figure 5 shows the pseudocode of the Halfmoon-read protocol. At the start of execution, the SSF appends an *init* log record, and uses the seqnum of this record as the initial *cursorTS* (line 7). The *cursorTS* is recovered from the log record, if present (line 5). The SSF also retrieve all records from the log stream tagged by its *instanceID* (line 3), which

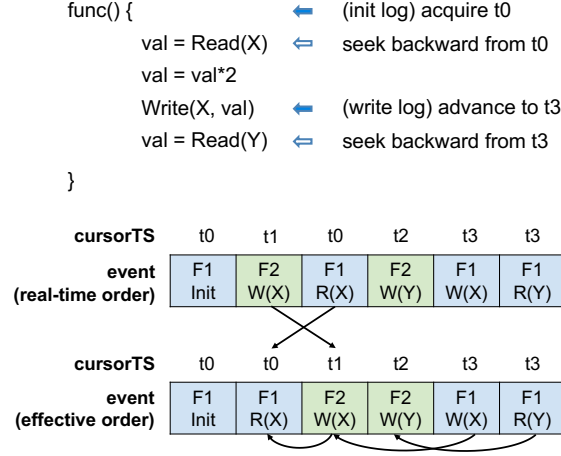


Fig. 4. Example of the Halfmoon-read protocol. F1 runs the pseudocode. F2 is another SSF. The top and bottom timeline show the real-time order and the effective order of events, respectively. The arrows below the bottom timeline show the dependencies between operations on object X and Y.

contains the execution history of the SSF. We refer to this per-SSF log stream as the step log. The current records in the step log are stored in a local array `env.stepLogs`. Later the SSF may check this array for existing log records and skip finished operations.

Write first obtains a version number (line 20), performs multi-version **DBWrite**, and finishes by logging the version number (line 22). Specifically, Halfmoon-read first checks if the write log record exists. If so, the write has been applied. Otherwise, it means that either the write has not yet created a new version of the object, or it has created the new version but crashed before logging.

To be idempotent, the write needs to use a deterministic version number. For example, it can generate the version number by simply concatenating the unique and deterministic InstanceID (`env.ID`) and the current step number (to distinguish different operations in the same SSF). Alternatively, if the version number is to be randomly generated, the SSF should log and check the version number *before* **DBWrite** to transform it into a deterministic operation. Our current prototype adopts the latter approach such that Halfmoon-read logs before and after **DBWrite**. This is because our primary baseline, Boki [52], also logs twice for each write. Our prototype aligns the logging overhead of writes such that our performance gains come solely from eliminating the logging of reads. Alternatively, if the external state exposes its internal logging and ordering of events, then **DBWrite** and logging can be merged into a single step, where it atomically creates a new object version and receives a timestamp.

Read first queries the per-object write log tagged by **key**. It passes the `cursorTS` into `logReadPrev` to retrieve a particular write log record. The “version” attribute in the record points to the actual object the read should see (line 28). To facilitate this process, we pass two tags into `logAppend` when logging the write, namely the instanceID of the SSF (`env.ID`) and the **key** of the target object, such that the record is visible in two log streams, namely the SSF’s step log and the object’s write log. Therefore, a write log record serves a dual purpose. First, it checkpoints the progress of the initiating SSF, allowing the write to be skipped during re-execution by checking the step log. Second, it functions as the commit point of the write where it becomes visible to other SSFs in the write log. Because the logging layer assigns

```

1 def Init(env, input):
2     # retrieve all log records of the SSF
3     env.stepLogs = getStepLogs(env.ID)
4     if env.stepLogs[0] is not None:
5         env.cursorTS = env.stepLogs[0]["seqnum"]
6     else:
7         env.cursorTS = logAppend([env.ID], LogRecord{
8             "step": 0, "op": "init",
9             "data": input,
10        })
11    env.step = 0
12
13 def Write(env, key, value):
14     # check if write can be skipped
15     env.step += 1
16     if env.stepLogs[env.step] is not None:
17         env.cursorTS = env.stepLogs[env.step]["seqnum"]
18     return
19     # deterministically generate version number
20     vNum = getVersionNumber(env)
21     DBWrite(key, value, version=vNum)
22     env.cursorTS = logAppend([env.ID, key], LogRecord{
23         "step": env.step, "op": "write",
24         "version": vNum,
25    })
26
27 def Read(env, key):
28     writeLog = logReadPrev(key, env.cursorTS)
29     return DBRead(key, version=writeLog["version"])
30
31 def Invoke(env, funcName, input):
32     # check if invoke can be skipped
33     env.step += 1
34     if env.stepLogs[env.step] is not None:
35         env.cursorTS = env.stepLogs[env.step]["seqnum"]
36     return env.stepLogs[env.step]["result"]
37     # deterministically generate ID
38     ID = getUUID(env)
39     # ID is passed into callee's env
40     result = InvokeFunc(ID, funcName, input)
41     env.cursorTS = logAppend([env.ID], LogRecord{
42         "step": env.step, "op": "invoke",
43         "result": result,
44    })

```

Fig. 5. Pseudocode of the Halfmoon-read protocol.

monotonically increasing seqnums, a read has full visibility of all writes with smaller logical timestamps, thereby allowing the read to seek backward in the event stream deterministically.

The second purpose requires that the logging be performed after `DBWrite`, as opposed to write-ahead logging. The reason is that the logging layer is decoupled from the external state. If the version number is made visible in advance, then reads could obtain version numbers with no matching objects in the external state. In contrast, Halfmoon-read ensures that the exposed object versions are always available.

In spite of being log-free, **Read** still needs to pay one round of **logReadPrev**. This overhead is implementation specific. Because Boki caches log records on function nodes, **logReadPrev** takes 0.12ms at the median and 0.72ms at the 99%-tile [52], which is negligible compared to **DBRead** (Table 1). Therefore our prototype of Halfmoon-read provides near-zero overhead for reads. Note that the critical data of a write log record consists only of its seqnum and version number, which can be covered in a few dozen of bytes. Therefore the cache size is not a concern here. Alternatively, if the logging layer is merged with the external state, a read can directly issue a query with a filter on object versions, instead of retrieving the version and the object separately.

Invoke first generates the callee’s instanceID (line 38), calls the function, and finishes by logging the result (line 41). In case the log record already exists, the actual invocation can be skipped. The callee’s instanceID must be deterministic to ensure idempotence. Similar to the version numbers in **Write**, the SSF can deterministically generate the instanceID from its context, or randomly generate it and perform additional logging and checking to turn it into a deterministic operation. In line with Boki, our prototype adopts the latter approach. Note that by logging the result, the SSF ensures that the cursorTS is monotonic even across invocations. Because each individual SSF is idempotent, by induction the entire workflow is also idempotent.

Remark. Halfmoon-read is primarily designed for key-value stores with read/write interfaces. To perform table-level queries, e.g., scan, join, and aggregation, one should first use **logReadPrev** to get a list of version numbers for all objects in the table. This list captures a snapshot of the table at a given timestamp. It is necessary because the ordering of individual writes is defined by the write log, and the version numbers are not ordered by themselves. As an optimization, it is possible to cache the database index in the logging layer to reduce the size of the returned list. Alternatively, the table should be read-only to bypass any logging or version lookup. Our prototype implementation of Halfmoon-read does not support queries over *mutable* tables.

As Halfmoon-read requires multi-versioning, an important concern is the garbage collection and the storage overhead. We address this problem in §4.6 and §4.7, respectively.

4.1.2 Idempotence of Halfmoon-Read.

We now prove that Halfmoon-read provides idempotence for SSFs regardless of failure and re-execution. We start with the following definition and assumption.

DEFINITION 4.1. Context. *For a particular function instance, let context be the set of local variables containing: (1) function input; (2) key-value pairs retrieved so far from **Read** operation; (3) cursorTS.*

ASSUMPTION 4.2. Deterministic execution. *Let $F1, F2$ be two instances of an SSF invocation. If $F1$ and $F2$ have identical contexts, then they will be performing the same operation (**Read** or **Write**) with the same arguments in the next step.*

Assumption 4.2 states that SSF execution is deterministic based on its context. In practice, non-determinism (e.g., calling random number generators or reading the clock) can be made deterministic by logging the results and restoring them during re-execution. For the sake of clarity, we only consider read and write operations using the key-value API, which can be generalized to other mechanisms of accessing or modifying the external state.

We do not consider **Invoke** operations here because the case for nested or asynchronous SSFs can be reduced to a single, sequential SSF. Specifically, the SSF operation sequence (with loops unrolled) can be represented as a directed acyclic graph (DAG). An asynchronous invocation creates a new branch, running concurrently with the main branch;

synchronous invocations stay in the main branch. To show that the entire DAG is idempotent, we only need to prove the case for every sequential path from the root. We now have the following theorem for Halfmoon-read.

THEOREM 4.3. *Operation sequences are identical across instances.* *Let $F1$ and $F2$ be two instances of an SSF invocation. Let $\{S_i^1\}_{1 \leq i \leq n}$ and $\{S_i^2\}_{1 \leq i \leq m}$ be the sequence of operations performed by $F1$ and $F2$, respectively. Then $S_i^1 = S_i^2$ for $1 \leq i \leq \min(n, m)$.*

PROOF. We prove the theorem by induction on the length of the common prefix of operations sequences. For the base case, we show that $F1$ and $F2$ have identical contexts after **Init** (Lemma 4.4). Then given Assumption 4.2, the prefix is identical for length one. We then show that **Write** or **Read** in Halfmoon-read always renders the same context given the same arguments (Lemma 4.5 and Lemma 4.6), which extends the induction hypothesis to longer prefixes, completing the proof. $\square$

LEMMA 4.4. *Idempotence of Init.* *If $F1$ and $F2$ are two instances of an SSF invocation, then $F1$ and $F2$ have identical contexts after **Init**.*

PROOF. The fact that $F1$ and $F2$ correspond to the same invocation implies that their inputs are identical. Since the set of key-value pairs retrieved from **Read** is empty for both instances, it remains to show that the initial cursorTSs are identical. Given the definition of **Init** in §4.1.1, the SSF either recovers the cursorTS from an existing log record or appends a new one. Because we assume that the logging layer is idempotent (§4), in both cases, the cursorTS cannot diverge between $F1$ and $F2$. $\square$

LEMMA 4.5. *Idempotence of Write.* ***Write** in Halfmoon-read deterministically creates a version of the object in the external storage and deterministically advances the cursorTS in the context.*

PROOF. Based on Assumption 4.2, **Write** uses deterministic version number (§4.1.1) and value for the object, which implies that each version of the object is immutable once created. Because **logAppend** is idempotent, the seqnum of the write log record is deterministic. Therefore the cursorTS is also deterministic after the write operation. $\square$

LEMMA 4.6. *Idempotence of Read.* ***Read** in Halfmoon-read deterministically adds a key-value pair to the SSF context.*

PROOF. Given its definition in §4.1.1, **Read** seeks backward on the write log to find the latest write log record whose seqnum does not exceed the cursorTS. Because the logging layer is totally ordered and issues monotonically increasing seqnums globally, the retrieved write log record is deterministic. It remains to show that the associated version of the object in the external storage is deterministic. Based on Assumption 4.2, **Write** uses deterministic version number (§4.1.1) and value for the object, which implies that each version of the object is immutable once created, concluding the proof. $\square$

In conclusion, Theorem 4.3 guarantees that every instance of an SSF invocation performs exactly the same sequence of operations. Combining it with Lemma 4.5 further ensures that the write operations in that sequence are idempotent in terms of modifying the external state. Therefore, Halfmoon-read provides idempotence for SSFs.

4.2 Halfmoon-Write: the Log-Free Write Protocol

Overview. The Halfmoon-read protocol is ideal for read-intensive workloads. We now present the Halfmoon-write protocol that supports log-free writes. Similar to Halfmoon-read, the idea is to assign deterministic timestamps to log-free operations, but with roles reversed. Under Halfmoon-write, all reads are logged, while writes use the cursorTS.

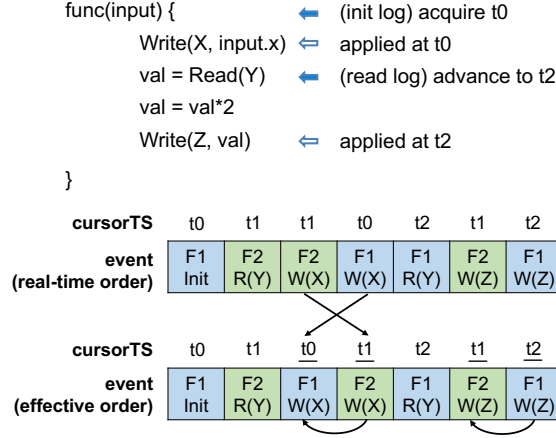


Fig. 6. Example of the Halfmoon-write protocol. F1 runs the pseudocode. F2 is another SSF. The top and bottom timeline show the real-time order and the effective order of events, respectively. The arrows below the bottom timeline show the dependencies between operations on object X and Z.

Because reads directly log the real-time data they have seen from the external state, they are idempotent on their own. Moreover, there is no need to use multi-versioning to keep the history of writes. Instead, writes in Halfmoon-write perform *conditional* updates to the *single-version* external state. Specifically, a write deterministically generates a version number based on the cursorTS, and updates the object only if the stored version number is smaller. The monotonicity of version numbers and the determinism of the cursorTS ensures that a write is always applied at the same point in the event stream, thereby achieving idempotence.

For writes, version numbers serve as their logical timestamps. Read timestamps, in contrast, are based on real time because reads always target the latest data. Note that the read timestamp is *implicit* and unknown to Halfmoon; SSFs have no need to explicitly parameterize reads with timestamps given the already *materialized* read log records. We define the read timestamps only to help understand the ordering of events in Halfmoon-write.

Example. Figure 6 shows an example of the Halfmoon-write protocol. F1 initializes its cursorTS to t_0 , and issues Write(X) using t_0 as the version number. At this point, F2 has already applied Write(X) with version number t_1 . Consequently, F1 does *not* overwrite F2’s Write(X) because its version number is smaller. The effect on the external state is equivalent to a virtual interleaving where F1’s Write(X) does “happen” before F2’s Write(X), which adheres to the definition of exactly-once semantics. In contrast, F1’s later Write(Z) overwrites F2’s Write(Z), in accordance with the real-time order. This is because F1 has advanced its cursorTS to t_2 after reading the latest value of Y.

Note that there is a major difference between Halfmoon-read and Halfmoon-write in terms of the ordering of events. In Halfmoon-read, the effective order of events follows the order of logical timestamps. In Halfmoon-write, logical timestamps (version numbers) are only relevant for writes, while reads are based on real time¹. Consequently, the effective order under Halfmoon-write combines the real-time and logical-timestamp order. We derive this order through the following steps. First, we order all events by real time. Second, for write events only, we reorder them according to their version numbers (§4.4).

¹Realtime-ness applies to failure-free reads. Since exactly-once semantics ensures that operations appear exactly once in the event stream, regardless of failure and re-execution, we consider only the failure-free case when discussing the ordering of events.

```

1 def Write(env, key, value):
2     env.consecutiveW += 1
3     vNum = (env.cursorTS, env.consecutiveW)
4     DBWrite(key, cond="VERSION < {vNum}",
5             update="VALUE={value}; VERSION={vNum}")
6
7 def Read(env, key):
8     env.step += 1
9     env.consecutiveW = 0
10    if env.stepLogs[env.step] is not None:
11        env.cursorTS = env.stepLogs[env.step]["seqnum"]
12        return env.stepLogs[env.step]["data"]
13    value = DBRead(key)
14    env.cursorTS = logAppend([env.ID], LogRecord{
15        "step": env.step, "op": "read",
16        "data": value,
17    })
18    return value

```

Fig. 7. Pseudocode of the Halfmoon-write protocol.

For example, Figure 6 underlines the version numbers of writes. F1’s Write(X) with version t_0 is ordered immediately before F2’s Write(X) with t_1 , but still *after* F2’s Read(Y). Formally, we show in §4.4 that the ordering under Halfmoon-write enforces a sequential history for each SSF except that consecutive log-free writes to different objects may commute. For now, we give an intuitive interpretation of Halfmoon-write’s reordering of writes. Because the cursorTS is refreshed after logging each read, a higher cursorTS implies that the SSF has seen “fresher” data, which in turn gives a higher priority to the SSF’s writes. In Figure 6, F1’s Write(X) is reordered because F2 has seen a fresher value of Y. F1’s later Write(Z) is not reordered because F1 is at least as fresh as F2.

4.2.1 The Halfmoon-Write Primitives.

Figure 7 shows the pseudocode of the Halfmoon-write protocol. It reuses the `Init` and `Invoke` functions from the Halfmoon-read protocol and differs only in `Read` and `Write`.

Write performs *conditional* update by comparing the version numbers. It is applied to the object only if the stored version number is smaller (line 4) [52]. The version number is structured as a tuple (line 3). The first field is the cursorTS; the second is a counter that records the number of *consecutive* writes. For simplicity, we omit the counter in Figure 6. The counter is incremented upon writes and reset upon reads. The purpose of the second field is to break ties between consecutive writes to the *same* object. A version number V_1 is smaller than V_2 if V_1 ’s cursorTS is smaller, or if they have equal cursorTS but V_1 ’s counter is smaller.

Read first recovers the previous result from the step log if possible. Otherwise, it reads the current object and logs the result. The cursorTS is updated accordingly. CursorTS is only relevant to subsequent log-free writes. Reads always see the latest state regardless of the cursorTS. Note that there is no per-object read log in Halfmoon-write, as opposed to the write log in Halfmoon-read. This is because read log records are only checked by the initiating SSF, so there is no need to tag them with the object’s key to make them publicly visible. Halfmoon-write only maintains the per-SSF step logs.

4.2.2 Idempotence of Halfmoon-Write.

We now prove that Halfmoon-write provides idempotence for SSFs regardless of failure and re-execution. Similar to Halfmoon-read, our goal is to show that Theorem 4.3 also applies to Halfmoon-write. Using the same inductive proof, it remains to validate the idempotence of **Write** and **Read** for Halfmoon-write. The difference between Halfmoon-read and Halfmoon-write in terms of write is that the latter does not change the SSF context. Based on Definition 4.1 and Assumption 4.2, we have the following lemmas.

LEMMA 4.7. Idempotence of Write. *Let $\{W_i\}_{i \in \mathbb{N}}$ be the sequence of writes applied to an object in the external storage under Halfmoon-write. Let $\text{src}(W_i)$ be a function that maps a write W_i to a particular operation in an SSF instance, i.e., $\text{src}(W_i) = (\text{instanceID}, \text{stepnumber})$. Then $\text{src}(W_i) = \text{src}(W_j)$ implies $i = j$.*

PROOF. Note that W_i must have succeeded in the conditional update; otherwise, it would be ignored by the external state. We prove the lemma by contradiction. Suppose there exists $\text{src}(W_i) = \text{src}(W_j)$ and $i < j$. By Assumption 4.2, W_i and W_j must have identical version numbers (essentially the cursorTS). Let it be v . Because $i < j$, the stored version number in the external state should be at least as high as v by the time W_j is applied. This contradicts the conditional update, concluding the proof. $\square$

LEMMA 4.8. Idempotence of Read. *Read in Halfmoon-write deterministically adds a key-value pair to the SSF context and deterministically advances the cursorTS in the SSF context.*

PROOF. While DBRead of different instances may not be the same, the read log record in the logging layer serves as a single source of truth. Because appending to the logging layer is assumed to be idempotent (§4), the read log record is guaranteed to be unique. Therefore the read result and the updated cursorTS must be deterministic. $\square$

In conclusion, provided that the sequence of operations is identical across instances, Lemma 4.7 further ensures that the external effect of each write is idempotent. Consequently, SSFs enjoy idempotence using Halfmoon-write.

4.3 Log Optimality

Given the two logging protocols that enable log-free exactly-once reads or writes, it is worth exploring whether there is still room for further optimization. We now prove that our protocols are log-optimal, i.e., no other log-based fault-tolerant protocol can achieve lower worst-case logging overhead than our protocols. Note that we only consider protocols that provide fault tolerance (i.e., idempotence) for *all* workloads over general mutable shared states. This implies that idempotence cannot be conditioned upon application semantics. For example, if reads in an application always target a read-only object, then there may be a more efficient protocol than Halfmoon that always skips logging these reads; however, this protocol cannot provide idempotence for all scenarios. Nevertheless, Halfmoon can utilize program analysis and verification tools to better understand application semantics and further optimize the logging overhead (§8).

We start by formalizing the concepts and assumptions. For simplicity, we focus on accessing a single object in this section.

DEFINITION 4.9. Write. *Let S be the set of valid states. A write operation is a function $w : S \rightarrow S$ that transforms the current state s to a new state $w(s)$.*

DEFINITION 4.10. Read. *A read is a function $r : S \rightarrow V$ that maps the current state s to a value $r(s) \in V$. A read is logged if there is a fault-tolerant record containing the read result.*

Without loss of generality, writes transform the current state while reads interpret it. Note that a read may not be defined over some states. For example, in Halfmoon-read, a read with cursorTS t is only valid if the latest seqnum in the logging layer is larger than t . A correct protocol must ensure at any time that any read allowed by the protocol is defined over the state at that time.

Next, we clarify the concept of logged and log-free writes. A write is logged if it is associated with a *standalone* record in fault-tolerant storage. Moreover, the record is either created by the write itself, e.g., in write-ahead logging, or it should be publicly visible, e.g. in Halfmoon-read. If there is no such record, the write is defined to be log-free. Consequently, in case a read is logged, we assume that the log record is private to the SSF; otherwise, it would be equivalent to associating a publicly visible log record with the write that created the object, so this write is also considered logged. Formally, we have the following assumption for log-free writes.

ASSUMPTION 4.11. *Log-free writes are memoryless.* *Let w be a log-free write over state s_1 with visible external effect. Then there exist $s_2 \in S$ and read r s.t. $w(s_1) = w(s_2)$ and $r(s_1) \neq r(s_2)$.*

Intuitively, a log-free write directly *overwrites* the object. It does not create a standalone record such that the old state is lost after the write. Consequently, there are multiple distinct old states that end with the same new state after the write. One cannot determine the actual old state from the new state alone. Note that the existence of r and s_2 entails that r is defined over both s_1 and s_2 and thus allowed by the protocol. Moreover, because $r(s_1) \neq r(s_2)$, at least one of them is not equal to $r(w(s_1))$, the read result over the current state. This implies that the write has visible external effects.

ASSUMPTION 4.12. *Arbitrary interleaving.* *SSFs are not aware of operations from others ahead of time, such that there can be arbitrary interleaving of reads and writes.*

Based on the definitions and assumptions, we have the following lemma that captures the relationship between the logging of reads and writes.

LEMMA 4.13. *If a fault-tolerant logging protocol allows an object to be updated with log-free writes (i.e., with visible external effect), then the protocol cannot be log-free on reads.*

PROOF. We prove by contradiction. We construct a counterexample that violates the idempotence of reads. Suppose an SSF performs a log-free read r over external state s_1 and then crashes. During the crash failure, s_1 is modified by a log-free write w . When the SSF re-executes the read, it cannot always recover the previous result based on the modified state. This is because given Assumption 4.11, we can choose s_2 and r s.t. $w(s_1) = w(s_2)$ and $r(s_1) \neq r(s_2)$. Moreover, given Assumption 4.12, the chosen s_2 and r in the counter example is always possible to happen in the worst case. Consequently, it is impossible for the read to choose between $r(s_1)$ and $r(s_2)$ as the previous read result, violating idempotence. $\square$

THEOREM 4.14. *In the worst case, a fault-tolerant logging protocol either logs all reads or all writes with visible external effects.*

PROOF. As per Lemma 4.13, if there are writes that are log-free and have visible external effects, then the protocol must log all reads so as to avoid the counterexample (which is otherwise possible based on Assumption 4.12). If there are no such writes, then by definition all writes with visible external effects are logged. $\square$

Remark. We only consider writes with visible external effects because a protocol may skip logging some writes by making sure that none can read them. For example, a protocol may handle log-free writes like Halfmoon-write do, occasionally take a snapshot of the object, and serve all reads using the snapshots like Halfmoon-read do. This implies that only the writes captured by the snapshots are visible and logged; the rest are effectively *stateless* operations with no visible effects.

We also note that the concept of logging in Lemma 4.13 and Theorem 4.14 is a general abstraction for installing a standalone record in fault-tolerant storage, independent of implementation. The actual cost of logging, however, is implementation-specific. For example, the write logging in Halfmoon-read can be overlapped with execution, or merged with `DBWrite` if the external state exposes its internal logging and ordering of events. Nonetheless, all writes in Halfmoon-read are considered logged because they are all associated with persistent objects in the multi-version external storage at the least. In contrast, even if an SSF performs logging in `Init` under Halfmoon-read and then issues a log-free read, the preceding logging operation is not considered part of the read because the `Init` call is not even aware of such an operation. As we show in §4.4, the logging in `Init` serves to bring the cursorTS up-to-date, and is not necessary for idempotence. The initial cursorTS is only required to be deterministic, which can be inherited from the parent SSF, or if there is no such SSF, be arbitrarily out-of-date.

Halfmoon measures the logging overhead as the number of *abstract* logging operations. Given any particular implementation, a fault-tolerant logging protocol either logs more reads than Halfmoon-read or logs more writes than Halfmoon-write in the worst case. Therefore our protocols identify two minimums in the design space. The actual choice between them depends on the implementation and the workload. We present an analysis in §4.7 to quantify this decision.

4.4 Consistency

The primary goal of this paper is to explore the minimal logging overhead required for idempotence. In doing so, Halfmoon relaxes the real-time guarantees of linearizability [47] such that events can be persisted without logging. Specifically, Halfmoon-read provides *sequential consistency* (SC), which guarantees that events are totally ordered, and the order adheres to the program order of each process in the system. The ordering under Halfmoon-write is semantically equivalent to SC as long as consecutive writes to different objects can commute. Formally, we have the following propositions. We include TLA+ verification in Appendix A.

PROPOSITION 4.15. *Halfmoon-read orders events according to their logical timestamps. This ordering provides sequential consistency.*

PROOF. For simplicity, we consider only read and write events; nested SSFs can be flattened into a single SSF with auxiliary logging at the function boundaries. It follows that writes are already totally ordered by the logging layer according to the write timestamps, because the creation of a write log record serves as the commit point of the corresponding write where it becomes visible to other SSFs. Because log-free reads are mapped to writes by seeking backward on the write log, the effective interleaving of reads and writes is equivalent to the one produced by inserting the reads in the ordered sequence of writes based on read timestamps.

To see that the timestamp order provides sequential consistency, note that the cursorTS is monotonically increasing within each SSF. Suppose that there is a violation of program order, then there must be a timestamp inversion in the total order, contradicting the way it is constructed. $\square$

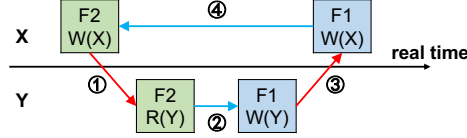
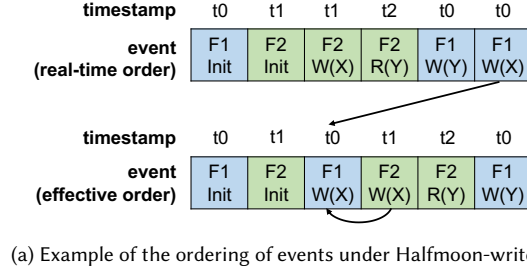


Fig. 8. Example of Halfmoon-write where consecutive log-free writes to different objects may commute.

PROPOSITION 4.16. *Halfmoon-write orders events through the following steps. First, all events are ordered by real time. Second, write events are reordered according to their version numbers. Specifically, a write is not reordered if it succeeds in conditional update (§4.2); otherwise, it is placed immediately before the next successful write to the same object with a higher version. This total ordering enforces a sequential history for each SSF except that consecutive log-free writes to different objects, i.e., those between two logged events, may commute.*

PROOF. In line with the proof of Proposition 4.15, we only consider read and write operations. All reads and all successful writes are processed in real-time in the external state, which naturally adheres to the actual interleaving of reads and writes. Writes that get reordered do not change the external state and are not visible to any other SSFs. The placement of these writes in the event stream should maintain the monotonicity of version numbers for each object, as a result of conditional updates, and also respect the data dependencies of reads. Therefore we insert a reordered write immediately before the next successful write to the same object with a higher version.

Taking the sequential history of each SSF from the total ordering, we note that reads and successful writes adhere to the program order because they are ordered by real time. It remains to show that a reordered write w may never go backward across a preceding read r in the SSF. Suppose w is placed immediately before w' , the next successful write. Then it suffices to show that w' happens later in real time than r . For w' to succeed in the conditional update, it must have acquired a higher cursorTS than that of w , which equals the one obtained after logging r . Because the logging layer issues monotonically increasing seqnums respecting the monotonicity of the real time, w' indeed happens later than r in real time. Note that by combining cursorTS with a local counter, version numbers are always monotonic within each SSF. Therefore consecutive log-free writes to the *same* object are never reordered. $\square$

Example. Figure 8a presents an example to illustrate the total ordering under Halfmoon-write. For simplicity, we consider only the cursorTS instead of the entire tuple for version numbers. F2's Write(X) with version t_1 and F1's Write(Y) with version t_0 succeed in conditional update and are not reordered. F1's Write(X) with version t_0 , however,

is reordered before F2's Write(X). Consequently, the program order of F1 is changed such Write(X) happens before Write(Y). However, although F1's Write(X) is reordered, it will never go past the preceding logged operation (Init in this case).

Ordering of consecutive writes. The example shows that Halfmoon-write may only change the program order of consecutive log-free writes to different objects. Consequently, the ordering of Halfmoon-write is exactly the same as SC when SSFs do not perform consecutive writes to different objects, and is semantically equivalent to SC when such writes can commute. In case the ordering of consecutive writes must be preserved, one can perform extra logging between the writes such that every dependent pair cannot be reordered. The best practice under Halfmoon-write is to make dependencies explicit through invocation or triggers in the SSF workflow, and utilize the logging in the init step of each SSF to prevent reordering. We observe this design pattern in a variety of workloads [3, 4, 9, 10, 32, 52, 119]. We also provide an extension of Halfmoon-write in §6 that preserves the ordering of consecutive writes. The extended protocol is log-free on writes in the best case.

Remark. One might wonder why the two protocols are dual to each other in terms of design but differ in the ordering of events. The reason is that writes have external effects while reads do not. Therefore, there is more freedom in placing log-free reads in the event stream than placing log-free writes. For a log-free read under Halfmoon-read, no matter when in real time the read is actually performed, we can always safely insert it in the event stream based on its assigned logical timestamp. In contrast, under Halfmoon-write, the placement of a log-free write not only depends on its version number (logical timestamp), but also on the real-time state of the object. If the conditional update succeeds, then the write must be placed precisely at this point in real time, since it would be immediately visible to reads after that point. This additional constraint leads to the permutation of program order in Figure 8.

Moreover, by advancing the cursorTS in *Init*, our protocols enforce a *real-time* property at the *boundary* of SSFs: if an operation finishes at t in real time, then all SSFs that starts after t are guaranteed to see the external effects of that operation. This property is well suited for a variety of serverless applications that use event triggers to invoke downstream tasks [5, 6, 8]. Optionally, an SSF can perform linearizable reads/writes by explicitly advancing the cursorTS beforehand. Similar to *Init*, the SSF appends a special *sync log* to acquire the up-to-date seqnum in the system. Halfmoon offers the flexibility for users to enforce linearizability if necessary, or achieve minimal logging overhead when our consistency guarantees suffice.

4.5 The Hybrid Protocol

So far, each protocol applies to accessing the entire external state. However, it is possible to use *independent* protocols per object, resulting in a hybrid protocol. This is because Halfmoon-read and Halfmoon-write differ only in the handling of reads and writes; they can share the same cursorTS of the SSF.

Idempotence. We note that the hybrid protocol is also idempotent. Because the cursorTS is still monotonically increasing within each SSF, we can directly use the inductive proofs of Halfmoon-read (§4.1.2) and Halfmoon-write (§4.2.2) to derive the idempotence of the hybrid protocol.

Consistency. The consistency of the Hybrid protocol is the weaker of Halfmoon-read and Halfmoon-write. We have the following proposition.

PROPOSITION 4.17. *The hybrid protocol orders events through the following steps. First, we order the following events by real time: (1) all writes of the Halfmoon-read; (2) all reads and writes from Halfmoon-write. Second, we reorder the*

log-free writes from Halfmoon-read as described in Proposition 4.16. Finally, we insert the log-free reads from Halfmoon-read according to their logical timestamp. Specifically, a read with timestamp t is inserted immediately after the logged event that receives seqnum t and advances the cursorTS to t . This total ordering enforces a sequential history for all logged events in each SSF; similar to Halfmoon-write, consecutive log-free operations may commute but never cross the surrounding logged events.

PROOF. For the first step, we note that the real time of a write in Halfmoon-write is determined by the point when the write log record is appended in the logging layer, i.e., its commit point. Because the logging layer issues globally monotonic seqnums, the ordering of writes in Halfmoon-write based on real time is the same as the original one based on logical timestamps (Proposition 4.15). Therefore we can safely align the ordering of Halfmoon-read and Halfmoon-write into a total ordering.

Second, we reorder the log-free writes from Halfmoon-read in the same way as Proposition 4.16. We now show that the reordering of log-free writes would never cross the surrounding logged events. We have already proved the case for logged reads in Halfmoon-write in the proof of Proposition 4.16. We now prove the case for writes in Halfmoon-read. We use lowercase w to denote log-free writes from Halfmoon-write, and uppercase W to denote logged writes from Halfmoon-read. Specifically, we wish to show that a reordered log-free write w may never go backward across a preceding logged write W in the SSF. Suppose w is placed immediately before w' , the successful log-free write that is applied after w . Then it suffices to show that w' happens later in real time than W . For w' to succeed in the conditional update, it must have acquired a higher cursorTS than that of w , which equals the one obtained after logging W . Because the logging layer issues monotonically increasing seqnums respecting the monotonicity of the real time, w' indeed happens later than W in real time.

Finally, the log-free reads in Halfmoon-read cannot be associated with real time. Therefore, we insert them based on their logical timestamps, relative to logged events that are also assigned logical seqnums. Note that the logged events can be writes from Halfmoon-read or reads from Halfmoon-write. The invariant is that the placement of a log-free read never goes past the preceding logged event in the SSF that advances the cursorTS to that used by the read; nor does it go past the succeeding one that advances the cursorTS to a higher value. After this step, the ordering of all reads and writes from Halfmoon-read is still monotonic concerning logical timestamps, identical to that in Proposition 4.15, which preserves the semantics of Halfmoon-read. $\square$

4.6 Garbage Collection

In line with previous work [52, 119], Halfmoon uses a garbage collector (GC) function to remove the log records of finished SSFs. The GC is periodically invoked. For Halfmoon-write, the lifetime of a read log record is equal to that of the initiating SSF. For Halfmoon-read, the GC should delete both the write log records and the matching object versions in the external state. Because a write log record has a dual purpose (§4.1), its lifetime should be the maximum of the SSF's and the object version's lifetime. Finally, the object version should outlive all SSFs that might read it.

Consequently, to garbage collect an object version whose matching write log record has seqnum t , the GC must wait until the following conditions are met: (a) there exists another record in the object's write log with seqnum $t' > t$, and (b) all SSFs that *starts before* t' , i.e., with initial cursorTS less than t' , finishes. Condition (b) entails the completion of the initiating SSF, so (a) and (b) also apply to garbage collecting the write log. Both conditions can be checked during the GC scan [52, 119]. Specifically, the GC tracks the latest seqnum $\bar{t}$ that satisfies (b). Upon advancing $\bar{t}$, for each per-object write log, the GC marks the latest log record whose seqnum falls below the new $\bar{t}$. These records point to the

earliest object versions that might still be observed by current or future SSFs. Therefore, for each per-object write log, it deletes all records preceding the marked records, as well as the corresponding object versions in the external state.

4.7 Choosing the Right Protocol

As described in §4.5, we can choose to use Halfmoon-read or Halfmoon-write on a per-object basis. Qualitatively, we should use the Halfmoon-read/write protocol for read/write-intensive objects, respectively. We now quantify the decision. For simplicity, we consider only read and write operations. Let P_r, P_w as the probability that an SSF reads and writes the object, respectively. Let λ be the average arrival rate of SSFs. Then we can express the read/write intensity by multiplying P_r or P_w with λ .

Storage overhead. Let t be the average function lifetime (including re-execution in case of failures). The lifetime analysis in §4.6 assumes that GC is performed as soon as possible. To account for the periodicity of GC, we define T_{gc} as the average delay between the completion of an SSF and the next GC scan.

We start by deriving the storage overhead for Halfmoon-write, which consists of the read log records and a single version of the object. Let N_r be the average number of read log records across time. By Little's Law [74], N_r equals the effective arrival rate of reads, which is $P_r\lambda$, times the average lifetime of read log records. We therefore have $N_r = P_r\lambda(t + T_{gc})$, and the time-averaged storage overhead S_{read} is

$$S_{read} = S_{val} + N_r(S_{meta} + S_{val}) \quad (1)$$

$$= S_{val} + P_r\lambda(t + T_{gc})(S_{meta} + S_{val}) \quad (2)$$

where S_{meta} and S_{val} denote the metadata size of a read log record and the object size, respectively. The full size of a read log record is $S_{val} + S_{meta}$.

For Halfmoon-read, the storage overhead consists of the write log and several versions of the object. Let N_w be the average number of write log records across time, which is also the average number of object versions. Let T_w be the average time gap between two consecutive writes to the object. According to §4.6, the average lifetime of a write log record and the corresponding object version is $T_w + t$, where T_w enforces condition (a), and t enforces (b). Considering T_{gc} , we have $N_w = P_w\lambda(T_w + t + T_{gc})$. Assuming a Poisson arrival of SSFs [67], we have $T_w = 1/(P_w\lambda)$. The average storage overhead S_{write} is

$$S_{write} = N_w(2S_{meta} + S_{val}) \quad (3)$$

$$= (1 + P_w\lambda(t + T_{gc}))(2S_{meta} + S_{val}) \quad (4)$$

Equation 3 assumes that the size of a write log is equal to S_{meta} . Note that there is a coefficient of two because our prototype of Halfmoon-read also logs *before* each write to align the write logging overhead with Boki (§4.1). We further assume that S_{meta} is negligible compared to S_{val} . Dividing both S_{read} and S_{write} with S_{val} , we derive the boundary condition as $P_r = P_w$. A higher read intensity means that Halfmoon-read has lower storage overhead, and vice versa.

Runtime overhead. Let C_w be the extra cost of a write in Halfmoon-read compared to that of Halfmoon-write. Similarly, let C_r be the extra cost of a read in Halfmoon-write over Halfmoon-read. The extra cost takes all relevant factors into account, including logging and multi-versioning. In a given time period T , the expected numbers of reads and writes to the object are $P_r\lambda T$ and $P_w\lambda T$, respectively. Then the expected extra costs of the two protocols are $P_w\lambda TC_w$ and $P_r\lambda TC_r$ in total, respectively. For our system prototype, we have $C_w \approx 2C_r$, where the coefficient of two is due to the same reason as that of Equation 3 (aligning our write logging overhead with Boki). The boundary

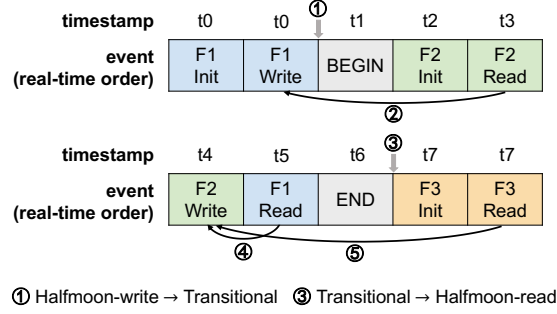


Fig. 9. Example of the switching procedure.

condition is then $P_r = 2P_w$. A higher read intensity means that Halfmoon-read has lower runtime overhead, and vice versa.

Remark. Note that the runtime analysis assumes that all SSFs have equal importance. To differentiate between SSFs, we can analyze the read and write activity of each SSF respectively, finally taking a weighted sum. We can also combine the runtime overhead with the storage overhead by taking another weighted sum, e.g., by their monetary cost, to facilitate the final decision.

4.8 Switching between Protocols

The intensity of read and write, namely P_r and P_w , may change over time for a particular object. Therefore, we design a switching procedure between the two protocols. Note that switching is a *global* decision over individual objects; the read or write intensity considered in this section is aggregated over all SSFs during a certain period.

We first provide an overview of the switching (§4.8.1). We then present a detailed description of the transitional read and write operations during the switching, utilizing a novel locking primitive to reconcile the single- and multi-version schemas under the two protocols (§4.8.2). Finally, we introduce a mechanism to optionally bump the external state up-to-date after the switching (§4.8.4).

4.8.1 Overview.

There are three major requirements. First, the switching must not violate Theorem 4.14. Second, the switching should not be a stop-the-world operation; SSFs should be able to run concurrently. Third, the switching must be fault-tolerant, i.e., SSFs must consistently use the same protocol for each step during re-execution.

To satisfy the above requirements, we maintain a *transition log* to record the switching history for each object. It is necessary because there can be an arbitrary delay between SSF failure and re-execution, possibly spanning several switching events. The runtime starts the switching by appending a “BEGIN” record to the transition log. It also scans the init log records (§4.1) to find all running SSFs that start before the switching. When all of these SSFs finish, the runtime completes the switching by appending an “END” record.

We now describe the switching from the perspective of an SSF. The first time an SSF reads or writes an object, it queries the transition log of the object to determine which protocol to use, and consistently uses the same protocol for that object in subsequent operations. The `SelectProtocol` in Figure 10 shows the pseudocode (line 1–6). Specifically, it calls `logReadPrev` to retrieve a transition log record, using the *initial* cursorTS acquired in `Init`. This ensures that the switching is fault-tolerant, since both the cursorTS and the transition log are persistent. If the log record is “END”,

the SSF should use the target protocol specified in the record as normal. However, if the record is “BEGIN”, the SSF should use a special *transitional* protocol that logs *all* reads and writes. It cannot use the new protocol immediately; otherwise, SSFs using the old protocol will run concurrently with those using the new, violating Theorem 4.14. Once all SSFs using the old protocol finish, the runtime can safely switch to the new protocol.

Concerning the external state, we need to support both single- and multi-versioning as required by Halfmoon-write and Halfmoon-read. Our solution is to treat single-versioning as a special case of multi-versioning. The former corresponds to a special LATEST version (managed by Halfmoon-write) among other versions (managed by Halfmoon-read). As per §4.1, multi-versioning can be implemented over plain key-value APIs where each version is represented by a separate key. Therefore the two versioning schemas can be seamlessly integrated. There is no intersection between them except during the switching.

Example. Figure 9 shows an example where the runtime switches from Halfmoon-write to Halfmoon-read. F1 (using Halfmoon-write) and F3 (using Halfmoon-read) perform single- and multi-version reads and writes, respectively. The handling of F2 is more complicated. F2’s write should be visible to both F1 and F3 (④ and ⑤), so it has to modify the LATEST version as well as create a standalone version. F2’s read should target both the LATEST version using Halfmoon-write (②) as well as some other version using Halfmoon-read (not shown in the figure), because its lifetime may overlap with both F1 and F3. F2 should compare the freshness of the data returned by the two protocols, namely the “version” attribute of the LATEST object version, and the seqnum of the write log record matching the other version. The fresher one is chosen as the read result and logged for idempotence.

4.8.2 Transitional Reads and Writes with Visibility Lock.

We now present the details of the transitional reads and writes during the switching. Figure 10 shows the pseudocode. We redesign the **Read** and **Write** operation to handle the transition between Halfmoon-read and Halfmoon-write. We introduce a novel locking primitive, the *visibility lock*, to atomically commit single- and multi-version writes.

Write with visibility lock. Transitional writes during the switching (**TransWrite**) should modify the LATEST version (for Halfmoon-write) as well as create a standalone version (for Halfmoon-read). The problem is that the two versioning schemas have different commit points for writes. The LATEST version, being a normal object in the external state, is committed once the conditional write succeeds. The standalone versions, being unordered by themselves, are committed once the corresponding write log records are appended to the logging layer. The transitional write should ensure that the two commits are atomic; otherwise, a single write may appear at two different points in the event stream, violating exactly-once semantics.

Our solution is to introduce a new locking primitive for the LATEST version during the switching. The *visibility lock* is implemented as a hash map associated with the LATEST object version. It is designed to block reads when the object is being updated by transitional writes. **TransWrite** acquires the lock by inserting its identifier, i.e., the (env.ID, step number) tuple, into the hash map (line 17 in Figure 10). Then it sequentially performs a multi-version write (line 19–25) and a single-version write (line 27–29). The seqnum of the write log record from the multi-version write is assigned as the version number for the single-version write (line 27). Finally, it releases the lock by deleting its identifier from the hash map. Because the identifier of the write operation is unique, acquiring and releasing the lock is inherently idempotent.

Note that the commit point of the entire write coincides with that of the constituent multi-version write. The visibility lock “hides” the LATEST version from reads so as not to expose a partially committed state where the single-version

```

1 def SelectProtocol(env, key):
2     transLog = logReadPrev(key+"TRANSITION", getInitTS(env))
3     if type(transLog) is BEGIN:
4         env.protocol[key] = TRANSITIONAL
5     else: # is END
6         env.protocol[key] = transLog["protocol"]
7
8 def visLock(env, key, id):
9     DBWrite(key, update="insert {id} -> LOCK")
10
11 def visUnlock(env, key, id):
12     DBWrite(key, update="delete {id} -> LOCK")
13
14 def TransWrite(env, key, value):
15     env.step += 1
16     stepID = (env.ID, env.step)
17     visLock(env, key, stepID)
18     vNum = getVersionNumber(env)
19     # standalone version
20     # omit checking if the log record already exists
21     DBWrite(key+str(vNum), value)
22     env.cursorTS = logCondAppend([env.ID, key],
23                                 LogRecord{
24                                     "step": env.step, "op": "write",
25                                     "version": vNum,
26                                 }, condTag=[env.ID], condPos=[env.step])
27     # the LATEST version
28     vNum = env.cursorTS
29     DBWrite(key+"LATEST", cond="VERSION < {vNum}",
30             update="VALUE={value}; VERSION={vNum}")
31     visUnlock(env, key, stepID)
32
33 def TransRead(env, key):
34     env.step += 1
35     # standalone version
36     writeLog = logReadPrev(key, env.cursorTS)
37     vNum = writeLog["version"]
38     standalone = DBRead(key+str(vNum))
39     # the LATEST version
40     latest = DBRead(key+"LATEST")
41     # get the fresher data from the two versions
42     value = getHigherVersionOf(standalone, latest)
43     # omit checking if the log record already exists
44     env.cursorTS = logCondAppend([env.ID], LogRecord{
45         "step": env.step, "op": "read",
46         "data": value,
47     }, condTag=[env.ID], condPos=[env.step])
48     return value

```

Fig. 10. Pseudocode of the switching procedure.

part has yet to be applied. Note that we perform the multi-version write first because it allows us to parameterize the commit point using the seqnum. Should we perform single-version write first, the commit point would be based on the real time, making it impossible to commit on the multi-version side, which is ordered by logical timestamps. While the visibility of the LATEST version is delayed by the lock, the multi-version write becomes immediately visible

once committed, because its commit point extends to the entire transitional write. Because the write operations on the LATEST version, either from transitional writes during the switching or from log-free writes under Halfmoon-write, are not dependent on each other, Halfmoon-write can safely ignore the visibility lock during its log-free writes.

Read with visibility lock. For an SSF that starts during a switching procedure, its lifetime may overlap with SSFs both before and after the switching. Therefore, a transitional read **TransRead** should target both the LATEST version as well as some multi-version object. Because we perform the multi-version write prior to the single-version write in **TransWrite**, any uncommitted writes to LATEST version, if issued by a transitional write, are guaranteed to be present as a standalone object version. Therefore, the transitional reads have no need to block on the visibility lock. Instead, it first reads from some standalone version and then from the LATEST version (line 34–39), and compares the freshness of the data, i.e., the seqnum of the write log record associated with the standalone version and the version number of the LATEST version (line 41). The seqnum and the version number are comparable because the latter is based on cursorTS, which is in turn based on seqnums. The fresher data is chosen as the read result and logged for idempotency. Similarly, reads under Halfmoon-read can also bypass the visibility lock.

However, because reads under Halfmoon-write only target the LATEST version and may miss uncommitted writes, they must wait until the visibility lock is free, i.e., the hash map becomes empty. In practice, Halfmoon-write does not actually block its reads. Instead, a read is considered valid only if the hash map in the retrieved object is empty; a non-empty hash map indicates that the read should be retried later. To avoid starving these reads, we can borrow ideas from the well-studied reader-writer problem [84] to introduce a reader lock beside the visibility lock.

4.8.3 Idempotence and Consistency.

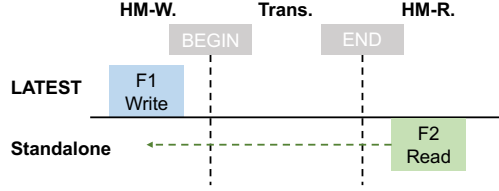
Idempotence. As described above, a **TransWrite** involves the following sequential steps: (1) acquiring the visibility lock, (2) performing a multi-version write as in Halfmoon-read, (3) performing a single-version write as in Halfmoon-write, and (4) releasing the visibility lock. Each step is idempotent and deterministically updates the SSF context. A **TransRead** is idempotent because it logs the read result. Therefore, the switching procedure retains idempotence.

Consistency. With the help of the visibility lock, the **TransWrite** is atomic despite performing a multi-version write and a single-version write. Therefore we can safely insert **TransWrite** and **TransRead** into the existing total order of events (§4.5). We have the following proposition.

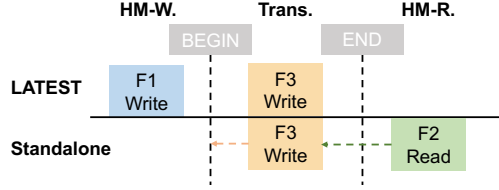
PROPOSITION 4.18. *Introducing the switching procedure retains the same level of consistency in Proposition 4.5.*

PROOF. Based on the way we construct the total ordering in Proposition 4.5, we insert the **TransWrites** between step one and step two. Specifically, because the commit point of a **TransWrite** is the same as that of the multi-version write it performs, we insert them in the same way as we insert the writes of Halfmoon-read, i.e., based on the real time of the commit point.

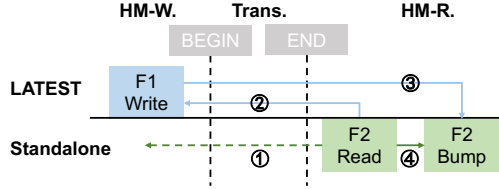
We insert the **TransReads** as a final step after the original step three. At this point all writes are already inserted. The position of a **TransRead** depends on the data it observes. If the data is written by a successful log-free write from Halfmoon-write, the **TransRead** is inserted based on its real time, the same way we insert the reads from Halfmoon-write. Otherwise, it is inserted based on its logical timestamp (Proposition 4.17), the same way we insert the reads from Halfmoon-reads. Note that despite using a mixture of real time and logical timestamps for insertion, consecutive **TransRead** in an SSF never gets reordered. This is because the logging of the read result always advances the cursorTS for subsequent usage of logical timestamps, in accordance with the real time.



(a) The external state could be stale after the switching.



(b) A read is up-to-date only if it maps to a write that took place no earlier than the beginning of the switching.



(c) A **BumpRead** operation (**BumpReadHMR** here) eagerly syncs between the LATEST version and standalone versions to bring the external state up-to-date after the switching.

Fig. 11. Example of switching from Halfmoon-write to Halfmoon-read.

In this total ordering, the placement of **TransRead** or **TransWrite** are all reduced to the placement of reads and writes from Halfmoon-read or Halfmoon-write, without introducing new dependencies. Therefore, the consistency of the hybrid protocol is retained. $\square$

4.8.4 Bringing the External State Up-To-Date After Switching.

The transitional reads and writes bridge the two Halfmoon protocols by targeting both versioning schemas. Nonetheless, the transitional SSFs that use such operations during the switching are only responsible for objects in their read and write sets. Suppose we are switching from Halfmoon-write to Halfmoon-read. Let there be an object that has been updated by Halfmoon-write before the switching ends (i.e., by SSFs that start before and overlap with the switching), but has not been touched by any transitional SSF (i.e., those that start after the switching). Then the latest state of that object is present in the LATEST version but not as any standalone version, thereby being invisible to Halfmoon-read after the switching (Figure 11a).

While this phenomenon is benign to the consistency of Halfmoon, which does not always enforce real-time reads and writes, it would be beneficial to bring the external state up-to-date after the switching. Following the previous example, for a log-free read after the switching, let $wlog$ be the write log record it retrieves in **logReadPrev** (Figure 5) during the read. Then the read result is considered up-to-date only if the seqnum of $wlog$ is higher than that of the latest

```

1 def BumpReadHMR(env, key):
2     # get the last BEGIN record in the transition log
3     transLogBegin = lastBeginFromTransLog(env)
4     # normal log-free read
5     writeLog = logReadPrev(key, env.cursorTS)
6     vNum = writeLog["version"]
7     standalone = DBRead(key+str(vNum))
8     # up-to-date
9     if writeLog["seqnum"] >= transLogBegin["seqnum"]:
10         return standalone
11     # possibly stale
12     env.step += 1
13     # at this point there is no SSF that uses HM-W to update the LATEST version,
14     # so ignore visibility lock
15     latest = DBRead(key+"LATEST")
16     # get the fresher data from the two versions
17     value = getHigherVersionOf(standalone, latest)
18     # create up-to-date standalone version
19     vNum = getVersionNumber(env)
20     DBWrite(key+str(vNum), value)
21     # conditional append to the write log; omit checking if the log record
22     # already exists
23     wLogPos = getWriteLogLengthAt(
24         tag=key, at=transLogBegin["seqnum"])
25     env.cursorTS, err = logCondAppend([env.ID, key],
26         LogRecord{
27             "step": env.step, "op": "write",
28             "version": vNum,
29         },
30         condTag=[key, env.ID],
31         condPos=[wLogPos, env.step])
32     if err is None:
33         return value
34     else:
35         return getObjFromConflict(err["seqnum"])

```

Fig. 12. Pseudocode of BumpHMR.

BEGIN record in the transition log, i.e., the one that initiates the transition from Halfmoon-write to Halfmoon-read (Figure 11b). This essentially requires that the read should target some write either from transitional SSFs (during the switching), or from SSFs using Halfmoon-read (after the switching). If there is no such write, then it is the responsibility of the read to create a standalone version that reflects the possibly fresher state in the LATEST version.

To this end, we extend the Read operation of Halfmoon-read and Halfmoon-write to **BumpRead**. Here we focus on **BumpReadHMR** as we are using Halfmoon-read after the switching (Figure 11c). The pseudocode of **BumpReadHMR** is shown in Figure 12. Specifically, it first performs a normal log-free read as in Halfmoon-read (① in Figure 11c, line 4–7 in Figure 12), and checks the seqnum of the write log record if it satisfied the "freshness" criterion (line 9 in Figure 12). If not, it proceeds to read from the LATEST version as Halfmoon-write would do, without the logging step (② in Figure 11c, line 14 in Figure 12). Then it performs a complete multi-version write as in Halfmoon-read to create a standalone version and the corresponding write log record (③ in Figure 11c, line 17–27 in Figure 12). The exception is that appending to the write log is conditional—the standalone version must map to the first log record in the per-object write log after the switching starts (line 21 in Figure 12). Otherwise, there could be a conflicting multi-version write that

```

1 def BumpReadHMW(env, key):
2     env.step += 1
3     # get the last BEGIN record in the transition log
4     transLogBegin = lastBeginFromTransLog(env)
5     # omit checking visibility lock
6     latest = DBRead(key+"LATEST")
7     # up-to-date
8     if latest["VERSION"] >= transLogBegin["seqnum"]:
9         value = latest
10    else:
11        # get standalone version
12        writeLog = logReadPrev(key, env.cursorTS)
13        vNum = writeLog["version"]
14        standalone = DBRead(key+str(vNum))
15        # get the fresher data from the two versions
16        value = getHigherVersionOf(standalone, latest)
17        # update the LATEST version
18        DBWrite(key, cond="VERSION < {vNum}",
19               update="VALUE={value}; VERSION={vNum}")
20    # append to the read log; omit checking if the read log record already exists
21    env.cursorTS = logCondAppend([env.ID], LogRecord{
22        "step": env.step, "op": "read",
23        "data": value,
24    }, condTag=[env.ID], condPos=[env.step])
25    return value

```

Fig. 13. Pseudocode of BumpHMW.

takes a higher priority over the bumped version. In the presence of a conflict, the `logCondAppend` should return the seqnum of the conflicting write log record along with an error message. `BumpReadHMR` would then return the up-to-date object version based on the conflicting record (④ in Figure 11c, line 31 in Figure 12).

In case we are switching from Halfmoon-read to Halfmoon-write, we use `BumpReadHMW` to bring the external state (the LATEST version) up-to-date (Figure 13). The logic of `BumpReadHMW` is simpler. It first fetches the LATEST version as in Halfmoon-read (line 6), and checks if it satisfied the "freshness" criterion (line 8). If not, it reads from a standalone version using a log-free read as in Halfmoon-read (line 11-14). Then it performs a conditional update on the LATEST version to apply the fresh data. By requiring that the version number be monotonically increasing on the LATEST object version, the conflict between the bumped version and concurrent writes, if any, is automatically resolved.

5 IMPLEMENTATION

We implement the Halfmoon prototype on top of Boki. We modify 2300 lines of C++ in the logging layer. Halfmoon's client library consists of 1700 lines of Go.

5.1 Resolving Conflicts Among Peer Instances

We discuss in §4 that there are two race conditions against the exactly-once semantics. We handle the first one in §4.1-4.2. To address the second one (the race between peer instances), We introduce `logCondAppend` (Figure 3). Compared with `logAppend`, it takes two additional parameters. `condTag` is a set of log tags that each maps to a specific log stream. For the sake of resolving conflicts, `condTag` is set to the caller SSF's instanceID (`env.ID`); the matching log stream contains the log records created by the SSF. `condPos` is the current step number of the caller. `logCondAppend` first

tries to append to the log normally as `logAppend` does. It then checks the offset of the log record in the caller’s log stream specified by `condTag`. The conditional append succeeds if the offset is equal to `condPos`, i.e., the step number is as expected and the log record appears in the right position in the SSF’s execution history. Otherwise, it undoes the log append and returns the seqnum of the log record at the expected offset, along with an error message. In case there are multiple `condTag` and `condPos` pairs, the log append is successful only if the record’s offset meets expectation on all log streams.

`logCondAppend` is similar to the compare-and-swap operation in concurrent programming. It resolves conflicts *in place* to ensure that only a single instance succeeds in logging a certain operation. For symmetric protocols, logging and conflict resolution can be performed *separately*. For example, Boki immediately reads the caller’s stream after appending to the log, and only honors the first log record of the current operation. This approach is practical because the sole purpose of logging in symmetric protocols is checkpointing SSF progress. Therefore the caller’s log stream is visible to the peer instances but not to other SSFs. Conflict resolution, though as a separate step, naturally serves as a synchronization point among peers, i.e., it ensures that all instances have identical states after this step. However, Halfmoon’s use of the write log (Figure 5), presents a new synchronization problem. Because the write log records also appear in the object’s log stream specified by the `key` tag, they are visible to other SSFs as well. Suppose we resolve conflicts separately, then a concurrent log-free read might see an inconsistent state of the object’s stream. In contrast, `logCondAppend` greatly simplifies the reasoning about concurrent instances.

We extend the implementation in §4 to handle the race condition among peers by simply replacing all `logAppend` with `logCondAppend`. If the conditional append fails, we let developers decide how to handle the error. For example, the SSF instance can use the returned seqnum to read the log records at the expected offset, and proceed with an identical state as its peers. Alternatively, it can quit the race by exiting.

6 EXTENSIONS OF HALFMOON

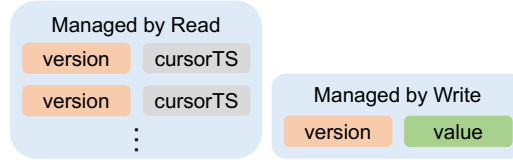
This section presents two extensions of Halfmoon. The first extension enforces strict ordering of consecutive writes, which provides sequential consistency. The second extension exports the current protocols to a partially ordered shared log, which generalizes Halfmoon to geo-distributed settings.

6.1 Strict Ordering of Consecutive Writes

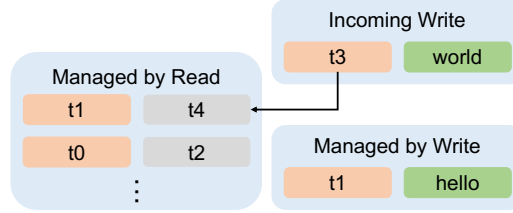
We design an extension of Halfmoon-write that eliminates possible reordering of consecutive writes (§ 4.4), thereby providing sequential consistency. The key idea is to utilize the pre-read cursorTS, i.e., the SSF’s cursorTS before logging the read result, to detect and break possible dependency cycles in Figure 8b. By definition, the pre-read cursorTS represents the highest possible version number this SSF might have used in its writes. This allows SSFs to check during writes if there exists a *timestamp inversion* edge, e.g., edge ② in Figure 8b, where a write has data dependency on a read with a higher cursorTS. If so, the write should perform extra logging to advance the cursorTS for subsequent writes, thereby inverting edge ④ in the dependency cycle.

The extended protocol is log-free on writes in the best case. By best case, we mean that whether the protocol requires extra write logging depends only on the actual interleaving of operations, regardless of the dependencies in the SSF code. In the best case, e.g., under a write-intensive workload, there is no timestamp inversion and the protocol can be log-free on writes.

To check for timestamp inversion, we let reads maintain a monotonically increasing sequence of (*version*, *pre-read cursorTS*) pairs for each object. Figure 14a shows the layout of an object under the extended protocol. The version field



(a) Object layout.



(b) A timestamp inversion where the write's version is lower than the pre-read cursorTS of a previous read.

Fig. 14. Example of the extended Halfmoon-write protocol.

is equal to the version of the object the read has seen. A read creates such a pair only if its cursorTS is higher than (i) the current object version, which indicates a potential timestamp inversion, and (ii) the largest pre-read cursorTS in the sequence of pairs, so as to ensure monotonicity. If several consecutive pairs have the same object version, they are merged by overwriting the tail of the sequence. Updating the sequence should be atomic with reading the object, which can be implemented using existing techniques such as DAAL [96, 119].

Upon visiting the object, a write atomically updates the object and searches for a pair in the sequence that has the largest object version below the version of the write. If the cursorTS of this pair is higher than the write version, then there is a timestamp inversion (Figure 14b). The subsequent extra logging step need not be atomic with the above step; the atomicity of reading the object and updating the sequence of pairs ensures idempotence in the detection of timestamp inversion. We provide a TLA+ verification of idempotence and sequential consistency of the extended protocol in Appendix A.

6.2 Halfmoon Over Partially Ordered Shared Log

The current design of Halfmoon depends on a totally ordered shared log, which can be costly to implement in geo-distributed settings [77, 101]. We present an extension of Halfmoon based on a partially ordered shared log, where users are clustered into different regions and each region has a locally ordered log. Because the logs across regions are only partially ordered, the extension provides the weaker but more scalable causal consistency [76] than the sequential consistency of original Halfmoon. Causal consistency only requires that causally dependent operations are delivered in the same order across regions, which allows us more flexibility in the design of the extension.

Halfmoon-read. Because a log-free read in Halfmoon-read (§ 4.1.1) seeks backward from its read timestamp on the write log and returns the latest preceding write. For such a read to be idempotent, we must maintain the invariant that once a read obtains t as its read timestamp, no subsequent writes can be ordered before t in the shared log. In the presence of a total order, this invariant is easily satisfied. With a partially ordered log, however, there is no restriction against the ordering of concurrent writes across regions. Fortunately, by targeting causal consistency, we

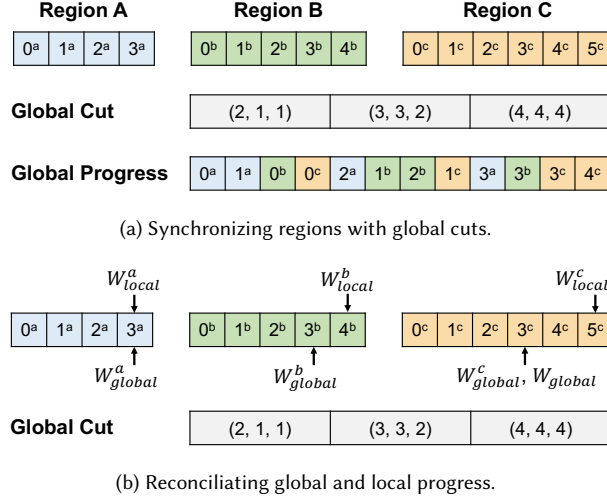


Fig. 15. Example of the extended Halfmoon-read protocol over a partially ordered log across three regions.

can instead restrict the scope of reads. That is, we let a read seek backward on the locally ordered write log within its region. Consequently, reads and writes from different regions are causally independent and can be arbitrarily ordered, eliminating the need for cross-region synchronization on the critical path. Another benefit of this design is that SSFs can still use the seqnum of its local log as cursorTS and read timestamps.

However, being causally consistent allows regions to diverge, even permanently. Therefore it is desirable to add a convergence property to causal consistency [76], i.e., being both causally and eventually consistent. Our solution is to periodically synchronize the logs across regions, but do so asynchronously without pausing reads or writes. Figure 15a shows the synchronization process. Similar to Boki's metalog [52], a global sequencer collects the progress of per-region logs and periodically publishes a cut. The increment between the two cuts indicates the global progress. To allow regions to converge, we should still order the log records within the global progress. However, the order can be arbitrary because records from different regions are causally independent. For example, we can simply order the records by the region ID, which allows each region to independently derive the same order.

Because the global cut is issued asynchronously, it could lag behind the local log. Each region then must reconcile the global progress with the local one. Let there be a read with timestamp t in region i . Let W_{local}^i denote the local log record it seeks backward to. Regarding the latest global cut received by region i as of timestamp t , let W_{global}^i denote the latest write log record from region i in that cut, and let W_{global} be the latest write log record from all regions after ordering the global progress. Then are two possible situations. If W_{global}^i does not lag behind W_{local}^i , then the read should seek to W_{global} . Otherwise, the read should seek to W_{local}^i . Figure 15b shows an example. Region A should use W_{global} , which is W_{global}^C , while Region B and C use W_{local}^B and W_{local}^C respectively. In simple terms, local writes generally take precedence over writes from other regions, in favor of causal consistency; but for "quiet" regions that are not ahead of the global progress, we respect the global progress in favor of convergence.

Note that Halfmoon-read relies on the shared log instead of the external store for the ordering of events. Moreover, because it uses multi-versioning, the external store is only required to be eventually consistent. If an object version is not found at the moment, Halfmoon-read should retry the read later.

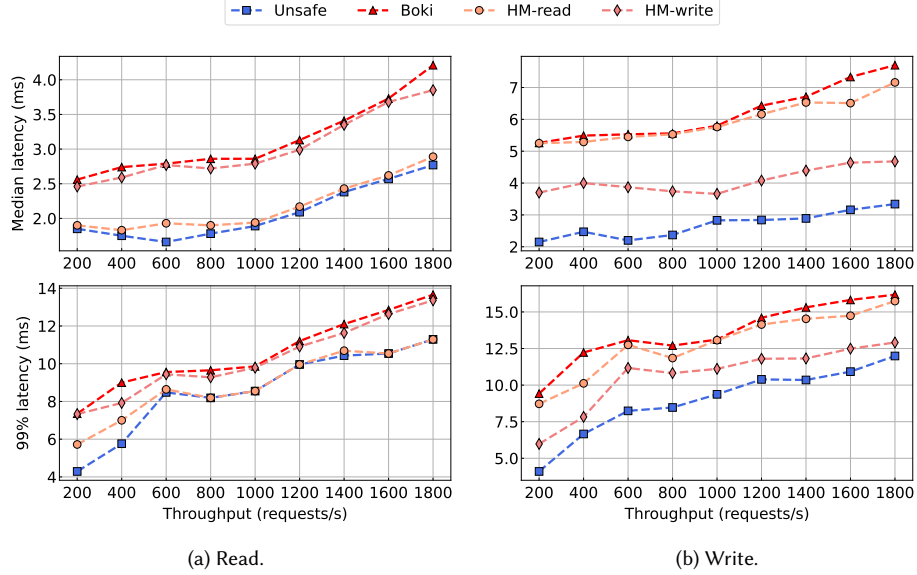


Fig. 16. Latency of read and write under different request rates. Halfmoon-read/write is abbreviated as HM-read/write.

Halfmoon-write. The extension of Halfmoon-write is more straightforward. Unlike Halfmoon-read, Halfmoon-write is mainly dependent on the external store for consistency. Consequently, the external storage should be causally consistent, which is known to be practical and scalable for geo-distributed settings [76]. Because external storage handles the ordering of writes from different regions, it remains for Halfmoon-write to enforce idempotence per region. Therefore the read log can be purely local. The only requirement is that the version field of an object should be a vector, where each element corresponds to a region [18, 66]. The conditional update in the write operation only needs to check the respective region’s version.

7 EVALUATION

This section compares Halfmoon’s performance with the state-of-the-art solution Boki [52], using microbenchmarks (§7.1) and realistic applications (§7.2). We also include an unsafe baseline with no logging. It does not offer exactly-once semantics and serves as the lower bound of Halfmoon. We evaluate Halfmoon’s storage and runtime overhead under different read/write intensity in §7.3, and explores the switching delay between Halfmoon’s protocols in §7.4.

Experimental setup. We conduct all our experiments on AWS EC2 c5d.2xlarge instances using the configuration reported in Boki [52]. Each instance has 8 vCPUs, 16GiB of DRAM, and 200GiB NVMe SSD. The runtime infrastructure consists of eight function nodes and one function gateway; the logging layer consists of three storage nodes and one sequencer node. Both Boki and Halfmoon use Amazon DynamoDB [36] as the external storage.

7.1 Microbenchmarks

We measure the median and 99%-tile tail latency of reads and writes over a period of 10 minutes under various request rates. In line with previous works [52, 119], we use a synthetic SSF that issues one read and write per request. We populate the external state with 10K objects, each consisting of 8B key and 256B value. Figure 16 shows the results. Compared to Boki, Halfmoon-read offers ~30% lower latency on reads and achieves similar performance on writes (as

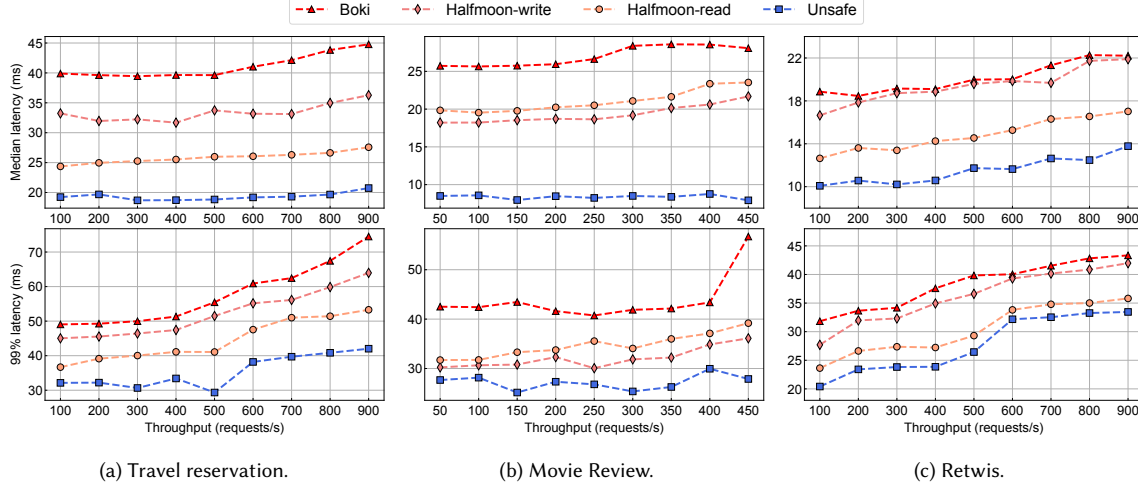


Fig. 17. End-to-end performance of Boki and Halfmoon under three application workloads.

per § 4.1, we deliberately align Halfmoon-read’s write logging overhead with Boki). It offers exactly-once reads with only $\sim 15\%$ overhead over unsafe raw reads, which is 4–5 $\times$ lower than Boki. Halfmoon-write also achieves $\sim 30\%$ lower latency than Boki on writes, and has similar performance on reads. The latency of log-free writes is still higher than raw writes, because Halfmoon-write performs *conditional* updates (§4.2), which is more expensive than directly updating the object. However, the overhead is still 2–4 $\times$ lower than Boki, of which writes are also conditional and require logging. The curves of median and tail latency show similar dynamics as the baselines, indicating that Halfmoon’s protocols do not introduce performance bottlenecks in the logging layer or the external storage.

7.2 End-to-End Application Workloads

We evaluate Halfmoon over three realistic application workloads. The first two workloads, travel reservation and movie review, are adapted from DeathStarBench [3, 42] and are commonly used in Boki and Beldi. Travel reservation runs a workflow of 10 SSFs. Users search for nearby hotels based on distance and ratings, and then make reservations. This workload is read-intensive. Movie review runs a workflow of 13 SSFs. It is slightly skewed towards writes as posting user reviews makes up its core functionality. The third workload is Retwis, a simplified Twitter clone [12]. Retwis consists of several Twitter functions (e.g., post tweet, get timeline) that perform PUTs and GETs on a key-value store. This workload is also read-intensive. All three workloads store application data in DynamoDB. We evaluate both Halfmoon-read and Halfmoon-write to demonstrate the benefits of using the appropriate protocol, as well as the worst-case performance penalties when using the wrong protocol.

Figure 17 shows the results. Using the appropriate protocol, Halfmoon offers 20%-40% lower median latency for the three workloads, and achieves 1.5–4.0 $\times$ lower overhead above the unsafe baseline. Logging is typically not the bottleneck of Boki, so it saturates at approximately the same load as Halfmoon. Halfmoon-read outperforms Halfmoon-write in read-intensive workloads (Figure 17a and 17c), and otherwise in write-intensive workload (Figure 17b). Halfmoon outperforms Boki even under the wrong protocol, because Boki either logs more reads than Halfmoon-read or logs more writes than Halfmoon-write.

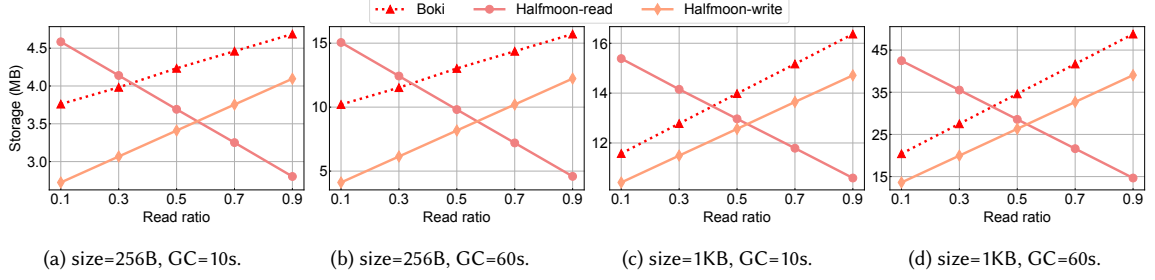


Fig. 18. Storage overhead of Boki and Halfmoon under different object size and GC interval.

7.3 System Overhead

We use a synthetic SSF to validate the overhead analysis in §4.7. The SSF issues 10 operations to the database. We vary the read and write intensity of the workload by changing the ratio of reads among the operations. We populate 10K objects in the database. Each operation in the SSF targets a random object, such that the read ratio is representative of the read intensity over each object. We compare the two protocols to determine the boundary condition when they have equal overhead. For reference, we also measure the overhead of Boki.

Storage overhead. We measure the time-averaged storage usage over a period of 10 minutes. The overall usage consists of log and database storage. Halfmoon-write stores a single version of each object, while Halfmoon-read stores multiple versions. We vary the size of each object and the GC interval. Figure 18 shows the results. §4.7 predicts that the boundary condition is when the read and write intensity are equal, i.e., the read ratio is 0.5. The theoretical boundary is an asymptotic result when the log storage is negligible compared with database storage. The actual boundary is slightly higher, because Halfmoon-read logs twice for each write (§4.1), while Halfmoon-write logs once for each read. As the object size increases, the boundary moves closer to 0.5 as the database storage becomes the decisive part. As per our analysis in §4.7, the boundary condition is not affected by the choice of GC interval in Figure 18. Halfmoon-read has a higher storage usage than Boki under low read ratio, i.e. high write intensity, because the overhead of multi-versioning outweighs that of the read log records, which are rather scarce. Compared with Boki, Halfmoon requires 1.2–3.4× less storage on average.

Runtime overhead. We measure the median latency under different request rates. The object size is set to 256B and the GC interval to 10s. Figure 19 shows the results. We predict in §4.7 that the boundary condition is when the read intensity is twice the write intensity, i.e., the read ratio is 2/3. The actual boundary is slightly higher, because C_w is more than twice C_r in practice (Figure 16). Figure 19 also confirms that the request rate has little impact on the boundary condition. Both of our protocols have lower latency than Boki, with an improvement factor of 1.2–1.5×. We empirically validate that Halfmoon’s runtime performance is insensitive to the GC interval. This is because the overhead of querying the log or database index typically scales logarithmically with the number of log records or object versions.

7.4 Switching Delay

We use the same SSF as §7.3 to evaluate the switching delay. The workload has two phases. The first phase runs Halfmoon-write with a read ratio of 0.2. The second runs Halfmoon-read with a read ratio of 0.8. We change the phase every five seconds. Figure 20 shows the dynamics of SSF latency over time. Under a moderate load of 300 requests/s,

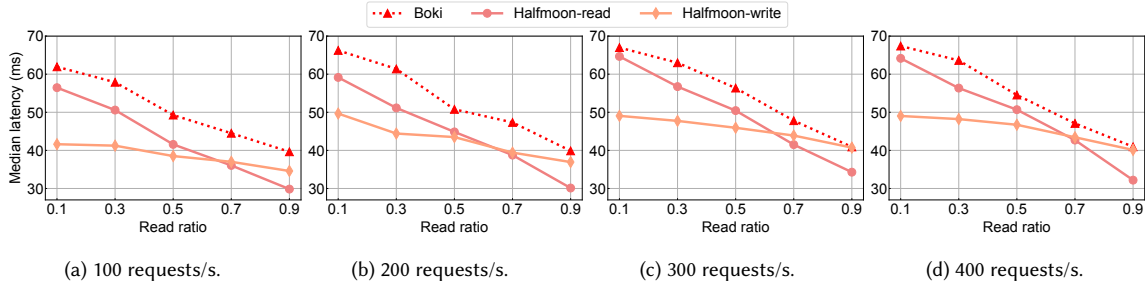


Fig. 19. Runtime overhead of Boki and Halfmoon under different request rates.

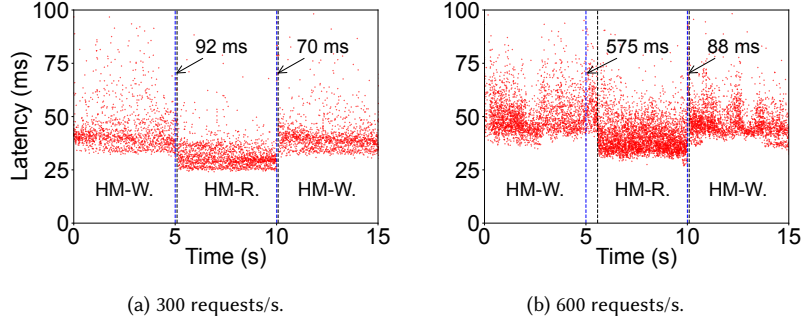


Fig. 20. The switching delay between Halfmoon's protocols (abbreviated as HM-R/W). The red dot shows SSF latency over time. The blue vertical line marks the beginning of switching. The black line marks its end.

the switching takes less than 100 ms. Under 600 requests/s (the workload saturates at about 800 requests/s), it takes longer to switch from Halfmoon-write to Halfmoon-read than the other way around. This is because SSFs in the first phase have a longer completion time due to the higher write intensity, and Halfmoon needs to wait until all SSFs using the old protocol finish before switching to the new protocol (§4.8).

7.5 Recovery Cost

Halfmoon's asymmetric protocols are designed for the failure-free common case. During re-execution, Halfmoon always replays log-free operations. In contrast, symmetric logging protocols such as Boki may skip logged operations. The speedup is implementation-specific. For Boki, the overhead of reading a log record is negligible compared to database operations. Therefore Boki always checks the logs before performing the actual read or write. In the extreme case, its recovery cost consists only of log reads.

We now incorporate the recovery cost into the end-to-end latency. We model the execution of an SSF as a Bernoulli Process. Let f be the probability of SSF failure. In each round, the SSF finishes with probability $1 - f$ and crashes with probability f . Then the expected rounds of execution n before the SSF returns successfully is $1/(1 - f)$. As an extreme case, we assume that the symmetric protocol has *zero* recovery cost, and the asymmetric protocol needs to replay *every* operation such that there is no speed up for recovery. Consequently, the total expected SSF latency under the symmetric protocols is simply the runtime cost of a single failure-free round. For our asymmetric protocols, the total expected latency consists of a single failure-free round and $n - 1$ recovery rounds (with the same runtime cost as the failure-free round). Suppose that our protocols have x percent less runtime cost than the symmetric protocols in the failure-free

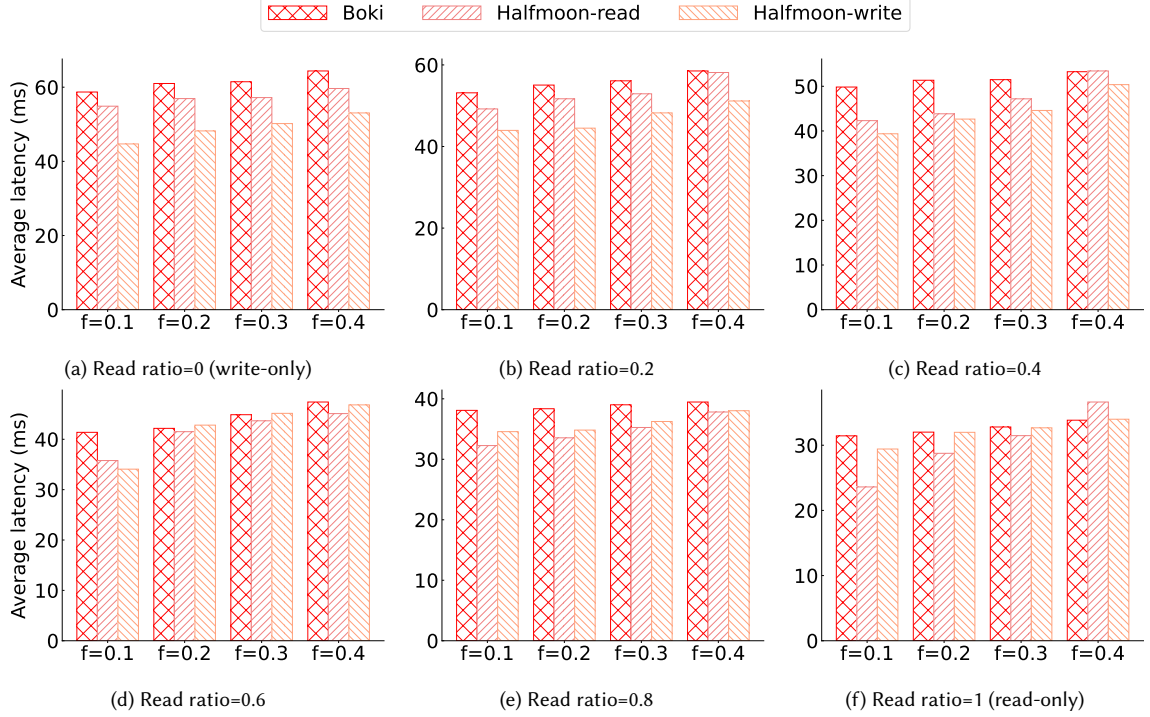


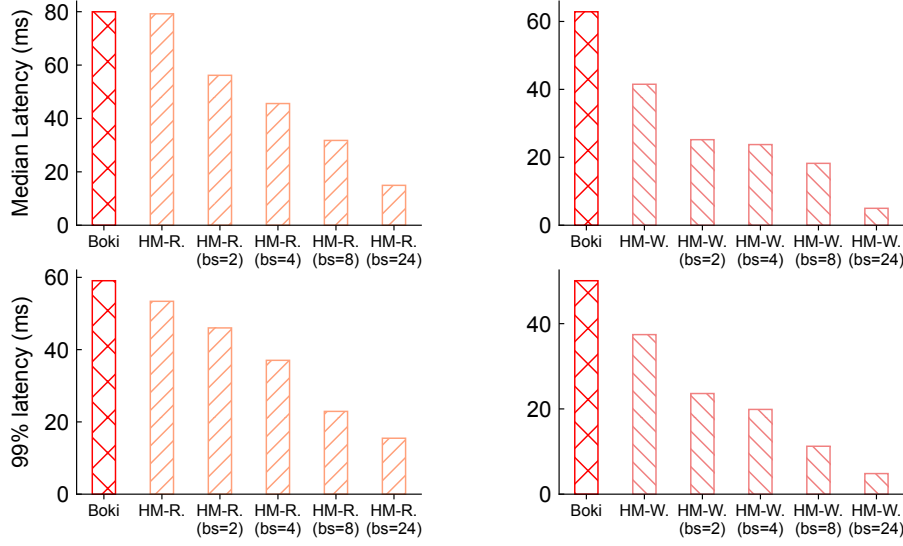
Fig. 21. The average end-to-end latency of Halfmoon and Boki under different failure rates.

case. Substituting x into the total expected latency, we see that the boundary condition is when f equals x , and a lower f implies that our asymmetric protocols have lower latency. According to Figure 16, the boundary condition between Boki and Halfmoon is $f \approx 30\%$, which far exceeds the actual failure rate of real applications.

We use the synthetic SSF workload in §7.3 to evaluate the end-to-end latency in the presence of failures. We run the workload at 100 requests/s and vary the read ratio of the workload from 0 to 1. Specifically, when the read ratio is 1, i.e., the workload is read-only, Halfmoon-read needs to replay every operation whereas Boki has approximately zero recovery cost. The same holds for Halfmoon-write when the read ratio is 0, i.e., the workload is write-only. Figure 21 shows the average latency of Halfmoon and Boki under different failure rates. Halfmoon has comparable performance to Boki up to 40% failure rate. As for the extreme cases, Halfmoon outperforms Boki when f is less than 30% and 40% under the read-only and write-only workloads, respectively.

7.6 Overlapping Logging with Execution

For the sake of clarity, the logging operations in Halfmoon's protocol are issued synchronously with reads or writes. In practice, SSFs do not need to wait for the completion of logging unless the current operation is dependent on the seqnum returned from a previous logging operation, i.e., logging can be overlapped with execution (§4.3). In Halfmoon-read, such dependency only involves write-read pairs. Because consecutive writes install standalone versions and the logging layer guarantees that log records are added following program order, SSFs can proceed without blocking on the logging of writes. The log-free reads, however, require the cursorTS to be at least as high as the seqnum of any preceding writes,



(a) Halfmoon-read under write-mostly workload (read ratio=0.2). (b) Halfmoon-write under read-mostly workload (read ratio=0.8).

Fig. 22. Logging overhead when overlapping logging with execution. We vary the batch size (bs) by which we check the completion of logging. Logging overhead is measured as the end-to-end latency minus that of the unsafe baseline.

such that they may fetch the up-to-date object version. Consequently, it suffices to ensure, as a pre-condition of issuing log-free reads, that all previous logging operations have finished, and the cursorTS is set to the highest seqnum of those. Similarly, in Halfmoon-write, only log-free writes are dependent on previous logging operations. Reads can proceed without waiting for logging operations to complete.

Overlapping logging with execution can further reduce the logging overhead. We also evaluate the end-to-end performance of Halfmoon under different degrees of overlapping. The implementation we have described earlier yields the highest degree of overlapping, i.e., only waiting for completion before issuing a log-free operation. We explore other degrees by issuing and waiting for a batch of logging operations. We use the workload in §7.3 where the SSF issues a series of reads and then writes. The total number of reads and writes is set to 30. To demonstrate the performance bonus of overlapping logging with execution, we explore a write-mostly and a read-mostly setting, using a read ratio of 0.2 (6 reads, 24 writes) and 0.8 (24 reads, 6 writes) respectively. As an adversary, we use the Halfmoon-read protocol for the write-mostly workload and Halfmoon-write for the read-mostly workload. We vary the batch size by which we wait for the completion of logging from two to eight, and the highest degree of overlapping is achieved at a batch size of 24. Figure 22 shows the logging overhead, i.e., the end-to-end latency minus that of the unsafe baseline, in line with the overhead measurement in §7.3. In spite of using the unsuitable protocol, overlapping can effectively reduce the logging overhead. Both protocols can match the performance of the unsafe baseline under a large batch size.

7.7 Overhead of the Halfmoon-write Extension

We evaluate the overhead of the extended Halfmoon-write protocol in §6.1 compared to the vanilla Halfmoon-write. We also measure the number of timestamp inversions (§6.1) to understand the cause of overhead and the necessity of the extension. We use the workload in §7.3, where an SSF issues 10 read or write operations to the database. We

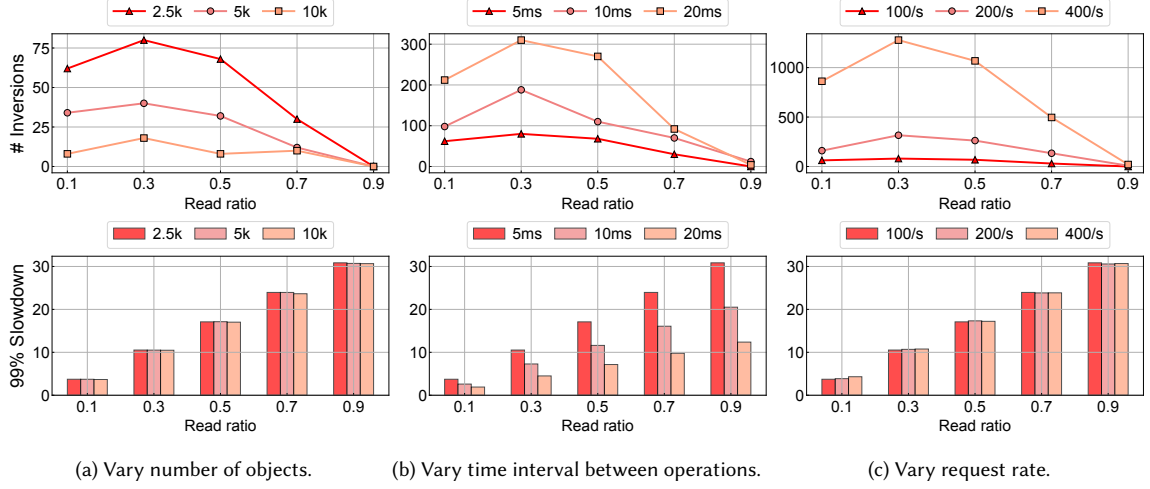


Fig. 23. Overhead of the extended Halfmoon-write protocol. The top row shows the average number of timestamp inversions per minute. The bottom row shows the relative slowdown of the extended protocol over vanilla Halfmoon-write.

change the ratio of reads among the operations to vary the read or write intensity. We explore three other dimensions of configurations that may affect the performance. First, we vary the number of objects in the database. A smaller number of objects increases the probability that an object is targeted by concurrent SSFs, resulting in more timestamp inversions. Second, we vary the time interval between reads and writes in an SSF. As demonstrated in the example in §8, the longer an SSF holds a stale cursorTS, the more likely it is to encounter a timestamp inversion when issuing a log-free write. Third, we vary the request rate, which also affects the probability of concurrent accesses and timestamp inversions. As the base case, we populate 2.5K objects, set the time interval to 5ms, and issue requests at 100/s. We vary the number of objects up to 10K, the time interval up to 20ms, and the request rate up to 400/s. The experiments run for 5 minutes.

The top row of Figure 23 shows the average number of timestamp inversions per minute. As expected, the number of inversions is negatively correlated with the number of objects, and positively with the time interval and the request rate. As for the read ratio, the number of inversions has a turning point. This is because it involves both a read and a write to the same object to trigger an inversion; the probability is higher when the read and write intensity are balanced. Under all configurations, the SSF encounters at most *one* inversion per invocation. This suggests that timestamp inversions are rare in practice.

The bottom row of Figure ?? shows the relative slowdown of the extended protocol over the vanilla Halfmoon-write. The extended **Read** adds 1.8–5.7ms overhead. The extended **Write** adds 0.5–1.6ms overhead alone, and an additional 1.1–2.0ms logging overhead if it encounters a timestamp inversion. Because **Read** is more expensive, the overall slowdown is positively correlated with the read ratio under all configurations. The number of objects and the request rate have little impact on the overhead, while a larger interval between operations amortizes the relative slowdown despite an increased number of timestamp inversions, since the extra logging overhead is not the dominant factor (at most one per SSF invocation). In the write-intensive workload that Halfmoon-write is intended for, the extended protocol adds 3–10% to the 99%-tile latency.

8 DISCUSSION

Ordering and timestamps. State machine replication (SMR) [14, 29, 37, 58, 61, 90] and its equivalents [19, 22, 24, 27, 43, 54, 78, 106] are common ways of building fault-tolerant distributed services [15]. SMR ensures that each process replicating the service executes commands following a global total order. Timestamp-based ordering is widely adopted in SMR. Global ordering can be established using synchronized clocks [28, 59] or timestamp agreement mechanisms [31, 37, 52]. Halfmoon’s event stream is built on top of SMR. It utilizes the event timestamps to deterministically parameterize log-free reads and writes. Halfmoon also requires that the event stream be traceable, which is naturally satisfied by existing SMR protocols that maintain a command log [19, 65, 89].

Security and privacy. The write log in Halfmoon-read serves a dual purpose (§4.7) and is visible to other SSFs. To avoid leaking private data, we can perform access control or encryption in the logging layer or the client library [122]. Only the shared fields should be exposed to other SSFs.

Program analysis and verification. Halfmoon takes no application-specific hints from SSF code. That is, it assumes the most pessimistic setup where all external operations are non-idempotent, which must be logged or deterministically parameterized. This is not necessarily true in practice. For example, if an object is read-only, then all reads to that object are inherently idempotent. Similarly, consider pushing to a message queue. If the receiver SSF can handle duplicate messages, then the push operation is also idempotent on its own. To avoid unnecessary logging, we can use existing tools [40, 41, 45, 49, 62, 69, 71, 80, 81, 86, 127] to analyze the SSF code beforehand to rule out such operations [69, 75, 79, 113, 114, 118]. Halfmoon is log-optimal in providing idempotent access to general mutable shared states.

Database logs. Logging is widely adopted in transactional databases [13, 88]. For example, write-ahead logging (WAL) is a common technique to ensure durability and atomicity [85, 96]. The WAL defines the order of state transitions in the database, which allows for multi-versioning, recovery, and rollback. The write logging in Halfmoon-read works in the same way to support multi-version deterministic log-free reads. In principle, Halfmoon can reuse the WAL in the database for idempotence. However, the WAL is usually internal to the database and not available to the application layer. This is why our underlying system, Boki, decouples the logs to a separate logging layer where it can explicitly order events with log seqnums. Alternatively, the database can expose its internal logs to inform the application whether it has already performed a certain operation, which could further reduce the overhead of providing exactly-once semantics.

Transactions. In line with previous works, we assume that SSFs are non-transactional by default [52]; to execute several steps atomically, SSFs should explicitly use transactions. Nevertheless, to enforce exactly-once semantics, it is indeed a viable approach to model the entire SSF as a transaction, which reduces the problem to atomicity [98]. However, it also implies that all external effects must be invisible to other SSFs until the transaction is committed, otherwise risking the *read uncommitted* anomaly [103]. Because most serverless functions do not require all of their external effects to be transactional [12, 52, 99, 119], the delayed visibility could potentially change the application semantics, especially for function triggers. For example, suppose function A writes to an object and continues its execution. If function B is to be triggered upon changes to that object, then B’s execution would be delayed until A is committed. While it is possible to insert early commit points in the function, it also involves manual effort. In contrast, we believe that providing idempotence at the operation granularity is a more flexible approach.

9 RELATED WORK

Stateful serverless computing on top of the stateless FaaS paradigm has been identified as a challenging task by many prior works [39, 46, 60, 99, 120]. A line of research optimizes the transfer of intermediate state across functions in analytic or stream processing workflows [60, 63, 72]. This paper targets a different scenario of sharing *mutable* state across SSFs. In terms of accessing the shared state, some works [52, 98, 99, 111, 119] adopt a data-oriented approach using table or key-value APIs, while others [2, 23, 25, 26, 30] adopt an object-oriented approach by encapsulating the state and access methods. Another line of research focuses on the formal semantics [26, 51, 57] and verification of SSFs [16, 32]. Halfmoon studies the orthogonal problem of the minimally necessary logging overhead for idempotence. It is an interesting direction to integrate Halfmoon’s idea with these works to automatically generate log-optimal SSF implementations.

Log-based replay is a well studied approach for fault tolerance [33, 48, 92, 96, 105] and troubleshooting [40, 56, 82, 93, 104, 116, 117, 124, 125, 128]. The idea has been broadly embraced by serverless computing [52, 57, 119]. Olive [96] proposes to use write-ahead redo logging to achieve exactly-once semantics. Beldi [119] extends Olive to the serverless environment and supports transactional workflows. Boki [52] implements Beldi’s techniques using the shared log [19, 31, 107]. In terms of reducing logging overhead, DDOS [50] enforces determinism in distributed systems and only requires logging message arrival times. Halfmoon has similar inspirations in stabilizing timestamps, and moves further to eliminate the logging overhead for either reads or writes.

Multi-versioning has been widely adopted in databases for concurrency control (MVCC) [87, 108, 112], where reads target old versions and writes install new versions. In the context of serverless computing, AFT [98] uses multi-versioning to provide read atomic isolation. The Halfmoon-read protocol applies the idea of existing works in a novel way to enable log-free exactly-once reads.

10 CONCLUSION

Halfmoon introduces novel optimizations to the log-based fault tolerance of SSFs. We design two logging protocols that enforce exactly-once semantics while providing log-free reads and writes, respectively. Instead of symmetrically logging every single read and write, our key insight is that it suffices to log *either* reads *or* writes, i.e., asymmetrically. We theoretically prove that our protocols are log-optimal. Therefore, Halfmoon pushes the overhead of log-based fault tolerance to its lower bound. We provide a criterion for choosing the right protocol for a given workload, and a pauseless switching procedure to switch protocols for dynamic workloads. Experiments show that Halfmoon achieves 20%–40% lower latency and 1.5–4.0× lower logging overhead than the state-of-the-art solution.

REFERENCES

- [1] 2023. AWS Step Functions. <https://aws.amazon.com/step-functions/>. Accessed 2023-04-17.
- [2] 2023. Azure Durable Entities. <https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-entities>. Accessed 2023-04-17.
- [3] 2023. DeathStarBench. <https://github.com/delimitrou/DeathStarBench/>. Accessed 2023-04-17.
- [4] 2023. Functionbench. <https://github.com/kmu-bigdata/serverless-faas-workbench>. Accessed 2023-04-17.
- [5] 2023. Google Cloud Functions Triggers. <https://cloud.google.com/functions/docs/calling>. Accessed 2023-04-17.
- [6] 2023. Logging in Azure Durable Functions. <https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-orchestrations>. Accessed 2023-04-17.
- [7] 2023. Retrying event-driven functions in Google Cloud. <https://cloud.google.com/functions/docs/bestpractices/retries>. Accessed 2023-04-17.
- [8] 2023. Sample projects for AWS Step Functions. <https://docs.aws.amazon.com/step-functions/latest/dg/create-sample-projects.html>. Accessed 2023-04-17.
- [9] 2023. Serverless Examples. <https://github.com/serverless/examples>. Accessed 2023-04-17.

- [10] 2023. Serverlessbench. <https://serverlessbench.systems/en-us/>. Accessed 2023-04-17.
- [11] 2023. Statelessness of Google Cloud Functions. <https://cloud.google.com/functions/docs/concepts/execution-environment>. Accessed 2023-04-17.
- [12] 2023. Tutorial: Design and implementation of a simple Twitter clone using PHP and the Redis key-value store. <https://redis.io/topics/twitter-clone>. Accessed 2023-09-11.
- [13] 2024. PostgreSQL. <http://www.postgresql.org/>. Accessed 2024-10-20.
- [14] Marcos K Aguilera, Naama Ben-David, Rachid Guerraoui, Virendra J Marathe, Athanasios Xygkis, and Igor Zablotchi. 2020. Microsecond consensus for microsecond applications. In *USENIX OSDI*.
- [15] Remzi Can Aksoy and Manos Kapritsos. 2019. Aegean: Replication beyond the Client-Server Model. In *ACM SOSP*.
- [16] Kalev Alpernas, Aurojit Panda, Leonid Ryzhyk, and Mooly Sagiv. 2021. Cloud-scale runtime verification of serverless applications. In *ACM Symposium on Cloud Computing*.
- [17] Mohamed Alzayat, Jonathan Mace, Peter Druschel, and Deepak Garg. 2023. Groundhog: Efficient Request Isolation in FaaS. In *EuroSys*.
- [18] Ozalp Babaoğlu and Keith Marzullo. 1993. Consistent global states of distributed systems: Fundamental concepts and mechanisms. *Distributed Systems* 53 (1993).
- [19] Mahesh Balakrishnan, Jason Flinn, Chen Shen, Mihir Dharamshi, Ahmed Jafri, Xiao Shi, Santosh Ghosh, Hazem Hassan, Aaryaman Sagar, Rhed Shi, et al. 2020. Virtual consensus in delos. In *USENIX OSDI*.
- [20] Mahesh Balakrishnan, Dahlia Malkhi, Vijayan Prabhakaran, Ted Wobber, Michael Wei, and John D Davis. 2012. CORFU: A Shared Log Design for Flash Clusters. In *USENIX NSDI*.
- [21] Mahesh Balakrishnan, Dahlia Malkhi, Ted Wobber, Ming Wu, Vijayan Prabhakaran, Michael Wei, John D Davis, Sriram Rao, Tao Zou, and Aviad Zuck. 2013. Tango: Distributed data structures over a shared log. In *ACM SOSP*.
- [22] Mahesh Balakrishnan, Chen Shen, Ahmed Jafri, Suyog Mapara, David Geraghty, Jason Flinn, Vidhya Venkat, Ivailo Nedelchev, Santosh Ghosh, Mihir Dharamshi, et al. 2021. Log-structured protocols in delos. In *ACM SOSP*.
- [23] Daniel Barcelona-Pons, Pierre Sutra, Marc Sánchez-Artigas, Gerard París, and Pedro García-López. 2022. Stateful serverless computing with crucial. *ACM Transactions on Software Engineering and Methodology* (2022).
- [24] Ken Birman and Thomas Joseph. 1987. Exploiting virtual synchrony in distributed systems. In *ACM SOSP*.
- [25] Sebastian Burckhardt, Badrish Chandramouli, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, Christopher S Meiklejohn, and Xiangfeng Zhu. 2022. Netherite: Efficient execution of serverless workflows. *Proceedings of the VLDB Endowment* (2022).
- [26] Sebastian Burckhardt, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, and Christopher S Meiklejohn. 2021. Durable functions: semantics for stateful serverless. In *ACM OOPSLA*.
- [27] Binbin Chen, Haifeng Yu, Yuda Zhao, and Phillip B Gibbons. 2014. The cost of fault tolerance in multi-party communication complexity. *J. ACM* (2014).
- [28] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2012. Spanner: Google’s globally-distributed database. In *USENIX OSDI*.
- [29] Heming Cui, Rui Gu, Cheng Liu, Tianyu Chen, and Junfeng Yang. 2015. Paxos made transparent. In *ACM SOSP*.
- [30] Martijn de Heus, Kyriakos Psarakis, Marios Fragkoulis, and Asterios Katsifodimos. 2021. Distributed transactions on serverless stateful functions. In *Proceedings of the ACM International Conference on Distributed and Event-based Systems*.
- [31] Cong Ding, David Chu, Evan Zhao, Xiang Li, Lorenzo Alvisi, and Robbert Van Renesse. 2020. Scalog: Seamless reconfiguration and total order in a scalable shared log. In *USENIX NSDI*.
- [32] Haoran Ding, Zhaoguo Wang, Zhuohao Shen, Rong Chen, and Haibo Chen. 2023. Automated Verification of Idempotence for Stateful Serverless Applications. In *USENIX OSDI*.
- [33] Zhiyuan Dong, Zhaoguo Wang, Xiaodong Zhang, Xian Xu, Changgang Zhao, Haibo Chen, Aurojit Panda, and Jinyang Li. 2023. Fine-Grained Re-Execution for Efficient Batched Commit of Distributed Transactions. (2023).
- [34] Dong Du, Qingyuan Liu, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2022. Serverless computing on heterogeneous computers. In *ACM ASPLOS*.
- [35] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *ACM ASPLOS*.
- [36] Mostafa Elhemali, Niall Gallagher, Bin Tang, Nick Gordon, Hao Huang, Haibo Chen, Joseph Idziorek, Mengtian Wang, Richard Krog, Zongpeng Zhu, et al. 2022. Amazon {DynamoDB}: A Scalable, Predictably Performant, and Fully Managed {NoSQL} Database Service. In *USENIX ATC*.
- [37] Vitor Enes, Carlos Baquero, Alexey Gotsman, and Pierre Sutra. 2021. Efficient replication via timestamp stability. In *EuroSys*.
- [38] Henrique Fingler, Zhiting Zhu, Esther Yoon, Zhipeng Jia, Emmett Witchel, and Christopher J Rossbach. 2022. DGSF: Disaggregated GPUs for Serverless Functions. In *IEEE International Parallel and Distributed Processing Symposium*.
- [39] Sadjad Fouladi, Francisco Romero, Dan Iter, Qian Li, Shuvo Chatterjee, Christos Kozyrakis, Matei Zaharia, and Keith Winstein. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient Functional Containers. In *USENIX ATC*.
- [40] Xinwei Fu, Wook-Hee Kim, Ajay Paddayuru Shreepathi, Mohannad Ismail, Sunny Wadkar, Dongyoon Lee, and Changwoo Min. 2021. Witcher: Systematic crash consistency testing for non-volatile memory key-value stores. In *ACM SOSP*.
- [41] Xinwei Fu, Dongyoon Lee, and Changwoo Min. 2022. {DURINN}: Adversarial Memory and Thread Interleaving for Detecting Durable Linearizability Bugs. In *USENIX OSDI*.

- [42] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *ACM ASPLOS*.
- [43] Aishwarya Ganesan, Ramnathan Alagappan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2021. Exploiting nil-externality for fast replicated storage. In *ACM SOSP*.
- [44] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiyang Zhang. 2022. Clio: A hardware-software co-designed disaggregated memory system. In *ACM ASPLOS*.
- [45] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2017. IronFleet: Proving Safety and Liveness of Practical Distributed Systems. *Commun. ACM* (2017).
- [46] Joseph M Hellerstein, Jose Faleiro, Joseph E Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. 2019. Serverless computing: One step forward, two steps back. (2019).
- [47] Maurice P Herlihy and Jeannette M Wing. 1990. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems* (1990).
- [48] Yige Hu, Zhiting Zhu, Ian Neal, Youngjin Kwon, Tianyu Cheng, Vijay Chidambaram, and Emmett Witchel. 2019. TxFS: Leveraging file-system crash consistency to provide ACID transactions. *ACM Transactions on Storage* (2019).
- [49] Peng Huang, Chuanxiong Guo, Jacob R Lorch, Lidong Zhou, and Yingnong Dang. 2018. Capturing and enhancing in situ system observability for failure detection. In *USENIX OSDI*.
- [50] Nicholas Hunt, Tom Bergan, Luis Ceze, and Steven D Gribble. 2013. DDOS: taming nondeterminism in distributed systems. *ACM SIGPLAN Notices* (2013).
- [51] Abhinav Jangda, Donald Pinckney, Yuriy Brun, and Arjun Guha. 2019. Formal foundations of serverless computing. (2019).
- [52] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful serverless computing with shared logs. In *ACM SOSP*.
- [53] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: efficient and scalable serverless computing for latency-sensitive, interactive microservices. In *ACM ASPLOS*.
- [54] Ricardo Jiménez-Peris, Gustavo Alonso, and Bettina Kemme. 2003. Are quorums an alternative for data replication? *ACM Transactions on Database Systems (TODS)* (2003).
- [55] Kostis Kaffes, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2019. Centralized Core-Granular Scheduling for Serverless Functions. In *ACM Symposium on Cloud Computing*.
- [56] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O'Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, et al. 2017. Canopy: An end-to-end performance tracing and analysis system. In *ACM SOSP*.
- [57] Konstantinos Kallas, Haoran Zhang, Rajeev Alur, Sebastian Angel, and Vincent Liu. 2023. Executing Microservice Applications on Serverless, Correctly. *ACM POPL* (2023).
- [58] Manos Kapritsos, Yang Wang, Vivien Quema, Allen Clement, Lorenzo Alvisi, and Mike Dahlin. 2012. All about eve: Execute-verify replication for multi-core servers. In *USENIX OSDI*.
- [59] Antonios Katsarakis, Vasilis Gavrielatos, MR Siavash Katebzadeh, Arpit Joshi, Aleksandar Dragojevic, Boris Grot, and Vijay Nagarajan. 2020. Hermes: A fast, fault-tolerant and linearizable replication protocol. In *ACM ASPLOS*.
- [60] Ana Klimovic, Yawen Wang, Patrick Stuedi, Animesh Trivedi, Jonas Pfefferle, and Christos Kozyrakis. 2018. Pocket: Elastic Ephemeral Storage for Serverless Analytics.. In *USENIX OSDI*.
- [61] Marios Kogias and Edouard Bugnion. 2020. HovercRaft: Achieving scalability and fault-tolerance for microsecond-scale datacenter services. In *EuroSys*.
- [62] Eric Koskinen and Junfeng Yang. 2016. Reducing crash recoverability to reachability. In *ACM POPL*.
- [63] Swaroop Kotni, Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. Faastlane: Accelerating Function-as-a-Service Workflows.. In *USENIX ATC*.
- [64] Leslie Lamport. 1979. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Trans. Comput.* (1979).
- [65] Leslie Lamport. 2001. Paxos made simple. *ACM SIGACT News* (2001).
- [66] Leslie Lamport. 2019. Time, clocks, and the ordering of events in a distributed system. In *Concurrency: the Works of Leslie Lamport*.
- [67] Günter Last and Mathew Penrose. 2017. *Lectures on the Poisson process*. Vol. 7. Cambridge University Press.
- [68] Sekwon Lee, Soujanya Ponnappalli, Sharad Singhal, Marcos K Aguilera, Kimberly Keeton, and Vijay Chidambaram. 2022. DINOMO: An Elastic, Scalable, High-Performance Key-Value Store for Disaggregated Persistent Memory. *Proceedings of the VLDB Endowment* (2022).
- [69] Guangpu Li, Haopeng Liu, Xianglan Chen, Haryadi S Gunawi, and Shan Lu. 2019. Dfix: automatically fixing timing bugs in distributed systems. In *ACM Conference on Programming Language Design and Implementation*.
- [70] Jialin Li, Ellis Michael, Naveen Kr Sharma, Adriana Szekeres, and Dan RK Ports. 2016. Just Say NO to Paxos Overhead: Replacing Consensus with Network Ordering.. In *USENIX OSDI*.
- [71] Jiaxin Li, Yiming Zhang, Shan Lu, Haryadi S Gunawi, Xiaohui Gu, Feng Huang, and Dongsheng Li. 2023. Performance Bug Analysis and Detection for Distributed Storage and Computing Systems. *ACM Transactions on Storage* (2023).
- [72] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. Faasflow: Enable efficient workflow execution for function-as-a-service. In *ACM ASPLOS*.

- [73] Barbara Liskov, Liuba Shrira, and John Wroclawski. 1991. Efficient at-most-once messages based on synchronized clocks. *ACM Transactions on Computer Systems* (1991).
- [74] John DC Little. 2011. Little’s Law as viewed on its 50th anniversary. *Operations Research* (2011).
- [75] Haopeng Liu, Guangpu Li, Jeffrey F Lukman, Jiaxin Li, Shan Lu, Haryadi S Gunawi, and Chen Tian. 2017. Dcatch: Automatically detecting distributed concurrency bugs in cloud systems. *ACM SIGARCH Computer Architecture News* (2017).
- [76] Wyatt Lloyd, Michael J Freedman, Michael Kaminsky, and David G Andersen. 2011. Don’t settle for eventual: Scalable causal consistency for wide-area storage with COPS. In *ACM SOSP*.
- [77] Joshua Lockerman, Jose M. Faleiro, Juno Kim, Soham Sankaran, Daniel J. Abadi, James Aspnes, Siddhartha Sen, and Mahesh Balakrishnan. 2018. The FuzzyLog: A Partially Ordered Shared Log. In *USENIX OSDI*.
- [78] Joshua Lockerman, Jose M. Faleiro, Juno Kim, Soham Sankaran, Daniel J. Abadi, James Aspnes, Siddhartha Sen, and Mahesh Balakrishnan. 2018. The FuzzyLog: A Partially Ordered Shared Log. In *USENIX OSDI*.
- [79] Jeffrey F Lukman, Huan Ke, Cesar A Stuardo, Riza O Suminto, Daniar H Kurniawan, Dikaimin Simon, Satria Priambada, Chen Tian, Feng Ye, Tanakorn Leesatapornwongsa, et al. 2019. Flymc: Highly scalable testing of complex interleavings in distributed systems. In *EuroSys*.
- [80] Haojun Ma, Hammad Ahmad, Aman Goel, Eli Goldweber, Jean-Baptiste Jeannin, Manos Kapritsos, and Baris Kasikci. 2022. Sift: Using Refinement-guided Automation to Verify Complex Distributed Systems. In *USENIX ATC*.
- [81] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem A Sakallah. 2019. I4: incremental inference of inductive invariants for verification of distributed protocols. In *ACM SOSP*.
- [82] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. 2015. Pivot tracing: Dynamic causal monitoring for distributed systems. In *ACM SOSP*.
- [83] Kostas Meladakis, Chrysostomos Zeginis, Kostas Magoutis, and Dimitris Plexousakis. 2022. Transferring transactional business processes to FaaS. In *Proceedings of the Eighth International Workshop on Serverless Computing*.
- [84] John M Mellor-Crummey and Michael L Scott. 1991. Scalable reader-writer synchronization for shared-memory multiprocessors. *ACM SIGPLAN Notices* (1991).
- [85] Chandrasekaran Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems (TODS)* 17, 1 (1992), 94–162.
- [86] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. 2019. Scaling symbolic evaluation for automated verification of systems code with Serval. In *ACM SOSP*.
- [87] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast serializable multi-version concurrency control for main-memory database systems. In *ACM SIGMOD*.
- [88] Michael A Olson, Keith Bostic, and Margo I Seltzer. 1999. Berkeley DB.. In *USENIX Annual Technical Conference, FREENIX Track*. 183–191.
- [89] Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm. In *USENIX ATC*.
- [90] Haochen Pan, Jesse Tuglu, Neo Zhou, Tianshu Wang, Yicheng Shen, Xiong Zheng, Joseph Tassarotti, Lewis Tseng, and Roberto Palmieri. 2021. Rabia: Simplifying state-machine replication through randomization. In *ACM SOSP*.
- [91] Sheng Qi, Xuanzhe Liu, and Xin Jin. 2023. Halfmoon: Log-optimal fault-tolerant stateful serverless computing. In *ACM SOSP*.
- [92] Dai Qin, Angela Demke Brown, and Ashvin Goel. 2017. Scalable replay-based replication for fast databases. *Proceedings of the VLDB Endowment* (2017).
- [93] Andrew Quinn, Jason Flinn, Michael Cafarella, and Baris Kasikci. 2022. Debugging the {OmniTable} Way. In *USENIX OSDI*.
- [94] Francisco Romero, Gohar Irfan Chaudhry, Íñigo Goiri, Pragna Gopa, Paul Batum, Neeraja J. Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. 2021. FaaS-T: A Transparent Auto-Scaling Cache for Serverless Applications. In *ACM Symposium on Cloud Computing*.
- [95] Johann Schleier-Smith, Vikram Sreekanti, Anurag Khandelwal, Joao Carreira, Neeraja J Yadwadkar, Raluca Ada Popa, Joseph E Gonzalez, Ion Stoica, and David A Patterson. 2021. What serverless computing is and should become: The next phase of cloud computing. *Commun. ACM* (2021).
- [96] Srinath TV Setty, Chunzhi Su, Jacob R Lorch, Lidong Zhou, Hao Chen, Parveen Patel, and Jinglei Ren. 2016. Realizing the Fault-Tolerance Promise of Cloud Storage Using Locks with Intent.. In *USENIX OSDI*.
- [97] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight isolation for efficient stateful serverless computing. In *USENIX ATC*.
- [98] Vikram Sreekanti, Chenggang Wu, Saurav Chhatrapati, Joseph E Gonzalez, Joseph M Hellerstein, and Jose M Faleiro. 2020. A fault-tolerance shim for serverless computing. In *EuroSys*.
- [99] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E Gonzalez, Joseph M Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful Functions-as-a-Service. *Proceedings of the VLDB Endowment* (2020).
- [100] Yang Tang and Junfeng Yang. 2020. Lambdata: Optimizing serverless computing by making data intents explicit. In *IEEE International Conference on Cloud Computing*.
- [101] Muhammed Uluyol, Anthony Huang, Ayush Goel, Mosharaf Chowdhury, and Harsha V Madhyastha. 2020. {Near-Optimal} Latency Versus Cost Tradeoffs in {Geo-Distributed} Storage. In *USENIX NSDI*.
- [102] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *ACM ASPLOS*.
- [103] Brecht Vandevoort, Bas Ketsman, Christoph Koch, and Frank Neven. 2023. When is it safe to run a transactional workload under Read Committed? *ACM SIGMOD Record* 52, 1 (2023), 36–43.

- [104] Kaushik Veeraraghavan, Dongyoon Lee, Benjamin Wester, Jessica Ouyang, Peter M Chen, Jason Flinn, and Satish Narayanasamy. 2012. DoublePlay: Parallelizing sequential logging and replay. *ACM Transactions on Computer Systems* (2012).
- [105] Stephanie Wang, John Liagouris, Robert Nishihara, Philipp Moritz, Ujval Misra, Alexey Tumanov, and Ion Stoica. 2019. Lineage stash: fault tolerance off the critical path. In *ACM SOSP*.
- [106] Zhaoguo Wang, Changgeng Zhao, Shuai Mu, Haibo Chen, and Jinyang Li. 2019. On the Parallels between Paxos and Raft, and how to Port Optimizations. In *ACM PODC*.
- [107] Michael Wei, Amy Tai, Christopher J Rossbach, Ittai Abraham, Maithem Munshed, Medhavi Dhawan, Jim Stabile, Udi Wieder, Scott Fritch, Steven Swanson, et al. 2017. vcorfu: A cloud-scale object store on a shared log. In *USENIX NSDI*.
- [108] Xingda Wei, Rong Chen, Haibo Chen, Zhaoguo Wang, Zhenhan Gong, and Binyu Zang. 2021. Unifying Timestamp with Transaction Ordering for MVCC with Decentralized Scalar Timestamp. In *USENIX NSDI*.
- [109] Xingda Wei, Fangming Lu, Tianxia Wang, J Gu, Y Yang, R Chen, and H Chen. 2023. No provisioned concurrency: Fast RDMA-codedesigned remote fork for serverless computing. (2023).
- [110] Jinfeng Wen, Zhenpeng Chen, Xin Jin, and Xuanzhe Liu. 2023. Rise of the Planet of Serverless Computing: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* (2023).
- [111] Chenggang Wu, Vikram Sreekanti, and Joseph M Hellerstein. 2020. Transactional causal consistency for serverless computing. In *ACM SIGMOD*.
- [112] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An empirical evaluation of in-memory multi-version concurrency control. *Proceedings of the VLDB Endowment* (2017).
- [113] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh. 2022. {DuoAI}: Fast, Automated Inference of Inductive Invariants for Verifying Distributed Protocols. In *USENIX OSDI*.
- [114] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan. 2021. DistAI: Data-Driven Automated Invariant Learning for Distributed Protocols. In *USENIX OSDI*.
- [115] Tianyi Yu, Qingyuan Liu, Dong Du, Yubin Xia, Binyu Zang, Ziqian Lu, Pingchao Yang, Chenggang Qin, and Haibo Chen. 2020. Characterizing serverless platforms with serverlessbench. In *ACM Symposium on Cloud Computing*.
- [116] Ding Yuan, Hao-hui Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. Sherlog: error diagnosis by connecting clues from run-time logs. In *ACM ASPLOS*.
- [117] Ding Yuan, Soyeon Park, Peng Huang, Yang Liu, Michael M Lee, Xiaoming Tang, Yuanyuan Zhou, and Stefan Savage. 2012. Be conservative: Enhancing failure diagnosis with proactive logging. In *USENIX OSDI*.
- [118] Xinhao Yuan and Junfeng Yang. 2020. Effective concurrency testing for distributed systems. In *ACM ASPLOS*.
- [119] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and transactional stateful serverless workflows. In *USENIX OSDI*.
- [120] Tian Zhang, Dong Xie, Feifei Li, and Ryan Stutsman. 2019. Narrowing the Gap Between Serverless and its State with Storage Functions. In *ACM Symposium on Cloud Computing*.
- [121] Wen Zhang, Vivian Fang, Aurojit Panda, and Scott Shenker. 2020. Kappa: A programming framework for serverless computing. In *ACM Symposium on Cloud Computing*.
- [122] Wen Zhang, Eric Sheng, Michael Chang, Aurojit Panda, Mooly Sagiv, and Scott Shenker. 2022. Blockaid: Data Access Policy Enforcement for Web Applications. In *USENIX OSDI*.
- [123] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and cheaper serverless computing on harvested resources. In *ACM SOSP*.
- [124] Yongle Zhang, Serguei Makarov, Xiang Ren, David Lion, and Ding Yuan. 2017. Non-Intrusive Failure Reproduction for Distributed Systems using the Partial Trace Principle. In *ACM SOSP*.
- [125] Yongle Zhang, Serguei Makarov, Xiang Ren, David Lion, and Ding Yuan. 2017. Pensieve: Non-intrusive failure reproduction for distributed systems using the event chaining approach. In *ACM SOSP*.
- [126] Ziming Zhao, Mingyu Wu, Jiawei Tang, Binyu Zang, Zhaoguo Wang, and Haibo Chen. 2023. BeeHive: Sub-second elasticity for web services with Semi-FaaS execution. In *ACM ASPLOS*.
- [127] Mo Zou, Haoran Ding, Dong Du, Ming Fu, Ronghui Gu, and Haibo Chen. 2019. Using concurrent relational logic with helpers for verifying the AtomFS file system. In *ACM SOSP*.
- [128] Gefei Zuo, Jiacheng Ma, Andrew Quinn, Pramod Bhatotia, Pedro Fonseca, and Baris Kasikci. 2021. Execution reconstruction: Harnessing failure reoccurrences for failure reproduction. In *ACM Conference on Programming Language Design and Implementation*.

Appendix A TLA+ SPECIFICATIONS

MODULE *HMBase*

EXTENDS *Naturals, Sequences*

CONSTANTS

SSFs,
NumPeers,
Objects,
MaxOps

VARIABLES

globalSeqnum,
ssfCommand,
ssfState,
ssfHistory,
objState,
objHistory

Types

EventIDs $\triangleq$ *SSFs* $\times$ (1 .. *MaxOps*) (id, pc)

InstanceIDs $\triangleq$ *SSFs* $\times$ (1 .. *NumPeers*) (id, pid)

Versions $\triangleq$ [*ts* : *Nat*, *pc* : 0 .. *MaxOps*]

SSFRecords $\triangleq$ [*pc* : 0 .. *MaxOps*,
opName : {"read", "write", "init"},
key : {""} $\cup$ *Objects*,
ts : *Nat*,
version : *Versions*]

SSFCommands $\triangleq$ [*opName* : {"read", "write"}, *key* : *Objects*]

SSFInternals $\triangleq$ [*cursorTS* : *Nat*,
pc : 0 .. *MaxOps*,
nextOp : {"init", "all", "readlog", "writelog"},
tmp : [*key* : {""} $\cup$ *Objects*,
version : *Versions*]]

ObjRecords $\triangleq$ [*eid* : *EventIDs*,
version : *Versions*,
opName : {"read", "write"}]

Type invariants

$$\begin{aligned}
SSFTypeOK &\triangleq \\
&\quad \wedge \text{ssfCommand} \in [SSFs \rightarrow Seq(SSFCommands)] \\
&\quad \wedge \text{ssfState} \in [InstanceIDs \rightarrow SSFInternals] \\
&\quad \wedge \text{ssfHistory} \in [SSFs \rightarrow Seq(SSFRecords)] \\
ObjHistoryTypeOK &\triangleq \text{objHistory} \in [Objects \rightarrow Seq(ObjRecords)] \\
GlobalTypeOK &\triangleq NumPeers \geq 1 \wedge globalSeqnum \in Nat
\end{aligned}$$

SSF invariants

$$\begin{aligned}
SSFCursorTSUnique &\triangleq \\
&\quad \forall id1, id2 \in SSFs, pid1, pid2 \in 1 \dots NumPeers : \\
&\quad \quad \text{LET } ts1 \triangleq \text{ssfState}[id1, pid1].cursorTS \\
&\quad \quad \quad ts2 \triangleq \text{ssfState}[id2, pid2].cursorTS \text{ IN} \\
&\quad \quad id1 \neq id2 \implies ts1 \neq ts2 \vee ts1 = 0 \vee ts2 = 0 \\
SSFCursorTSConsistent &\triangleq \\
&\quad \forall id \in SSFs, pid \in 1 \dots NumPeers : \\
&\quad \quad \text{ssfState}[id, pid].cursorTS \leq globalSeqnum \\
SSFHistoryMonotone &\triangleq \\
&\quad \forall id \in SSFs : \\
&\quad \quad \text{LET } h \triangleq \text{ssfHistory}[id] \text{ IN} \\
&\quad \quad \forall i, j \in 1 \dots Len(h) : \\
&\quad \quad \quad \wedge h[i].pc = i - 1 \\
&\quad \quad \quad \wedge i < j \implies h[i].ts \leq h[j].ts
\end{aligned}$$

Object invariants

$$\begin{aligned}
VersionHigher(left, right) &\triangleq \\
&\quad \vee left.ts > right.ts \\
&\quad \vee left.ts = right.ts \wedge left.pc > right.pc \\
VersionHigherOrEq(left, right) &\triangleq \\
&\quad \vee left = right \\
&\quad \vee VersionHigher(left, right) \\
ObjHistoryVersionMonotone &\triangleq \\
&\quad \forall key \in Objects : \\
&\quad \quad \text{LET } objh \triangleq \text{objHistory}[key] \text{ IN} \\
&\quad \quad \forall i, j \in 1 \dots Len(objh) : \\
&\quad \quad \quad i < j \implies VersionHigherOrEq(objh[j].version, objh[i].version)
\end{aligned}$$

$BaseInvariants \triangleq$
 $\wedge SSFTypeOK$
 $\wedge ObjHistoryTypeOK$
 $\wedge GlobalTypeOK$
 $\wedge SSFCursorTSUnique$
 $\wedge SSFCursorTSConsistent$
 $\wedge SSFHistoryMonotone$
 $\wedge ObjHistoryVersionMonotone$

Init formula

$InitGlobalSeqnum \triangleq globalSeqnum = 0$

$InitSSF \triangleq$

$\wedge ssfCommand = [id \in SSFs \mapsto \langle \rangle]$
 $\wedge ssfState = [id \in SSFs, pid \in 1 \dots NumPeers \mapsto$
 $\quad [cursorTS \mapsto 0,$
 $\quad pc \mapsto 0,$
 $\quad nextOp \mapsto \text{"init"},$
 $\quad tmp \mapsto [key \mapsto \text{"},$
 $\quad \quad version \mapsto [ts \mapsto 0,$
 $\quad \quad \quad pc \mapsto 0]]]]$
 $\wedge ssfHistory = [id \in SSFs \mapsto \langle \rangle]$

let child modules specify object state

$InitObjectHistory \triangleq objHistory = [key \in Objects \mapsto \langle \rangle]$

$BaseInit \triangleq$

$\wedge InitGlobalSeqnum$
 $\wedge InitSSF$
 $\wedge InitObjectHistory$

Next formula

$GetStepOffset(ssfh, pc, opName, key) \triangleq$

check that the step is unique and monotone instead

$LET\ offsets \triangleq \{i \in 1 \dots Len(ssfh) :$
 $\quad \wedge ssfh[i].pc = pc$
 $\quad \wedge ssfh[i].opName = opName$
 $\quad \wedge ssfh[i].key = key\}$ **IN**
 $IF\ offsets = \{\}$ **THEN** 0

```

ELSE CHOOSE  $i \in \text{offsets}$  : TRUE

 $\text{GetOrLogInitTs}(id, pid, \text{deltaTS}) \triangleq$ 
  LET  $\text{step} \triangleq \text{GetStepOffset}(\text{ssfHistory}[id], 0, \text{"init"}, \text{""})$ 
     $\text{initTS} \triangleq \text{globalSeqnum} + \text{deltaTS}$  IN
  IF  $\text{step} = 0$  THEN
     $\wedge \text{ssfState}' = [\text{ssfState} \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.\text{nextOp} = \text{"all"},$ 
       $!.cursorTS = \text{initTS}]]$ 
     $\wedge \text{ssfHistory}' = [\text{ssfHistory} \text{ EXCEPT } ![id] = \text{Append}(@, [pc \mapsto 0,$ 
       $opName \mapsto \text{"init"},$ 
       $key \mapsto \text{"},$ 
       $ts \mapsto \text{initTS},$ 
       $version \mapsto [ts \mapsto 0, pc \mapsto 0]])]$ 
     $\wedge \text{globalSeqnum}' = \text{initTS}$ 
  ELSE LET  $cursorTS \triangleq \text{ssfHistory}[id][\text{step}].ts$  IN
     $\wedge \text{ssfState}' = [\text{ssfState} \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.\text{nextOp} = \text{"all"},$ 
       $!.cursorTS = cursorTS]]$ 
     $\wedge \text{UNCHANGED } \langle \text{globalSeqnum}, \text{ssfHistory} \rangle$ 

 $\text{EntryPoint}(id, pid) \triangleq$ 
  LET  $\text{ssf} \triangleq \text{ssfState}[id, pid]$ 
     $\text{step} \triangleq \text{GetStepOffset}(id, 0, \text{"init"}, \text{""})$  IN
   $\wedge \text{ssf.pc} = 0$ 
   $\wedge \text{ssf.nextOp} = \text{"init"}$ 
   $\wedge \text{ssf.cursorTS} = 0$ 
   $\wedge \exists \text{deltaTS} \in 1 \dots \text{MaxOps} : \text{GetOrLogInitTs}(id, pid, \text{deltaTS})$ 
   $\wedge \text{UNCHANGED } \langle \text{objState}, \text{objHistory}, \text{ssfCommand} \rangle$ 

 $\text{ProposeCommand}(id, opName, key) \triangleq$ 
  LET  $\text{cmds} \triangleq \text{ssfCommand}[id]$  IN
   $\wedge \text{Len}(\text{cmds}) < \text{MaxOps}$ 
   $\wedge \text{ssfCommand}' = [\text{ssfCommand} \text{ EXCEPT } ![id] = \text{Append}(@, [opName \mapsto opName,$ 
     $key \mapsto key])]$ 
   $\wedge \text{UNCHANGED } \langle \text{globalSeqnum}, \text{ssfState}, \text{ssfHistory}, \text{objState}, \text{objHistory} \rangle$ 

 $\text{BaseNext} \triangleq$ 
   $\vee \exists id \in \text{SSFs}, \text{cmd} \in \text{SSFCmds} : \text{ProposeCommand}(id, \text{cmd.opName}, \text{cmd.key})$ 
   $\vee \exists \langle id, pid \rangle \in \text{InstanceIDs} : \text{EntryPoint}(id, pid)$ 

```

Helper functions

```

 $\text{Min}(\text{set}) \triangleq \text{IF } \text{set} = \{\} \text{ THEN } 0$ 

```

```

ELSE CHOOSE  $x \in set : \forall y \in set : x \leq y$ 

 $Max(set) \stackrel{\Delta}{=} \text{IF } set = \{\} \text{ THEN } 0$ 
ELSE CHOOSE  $x \in set : \forall y \in set : x \geq y$ 

returns the index of the last element that satisfies the predicate
 $Partition(array, len, lhs, boolOp(-, -)) \stackrel{\Delta}{=}$ 
  IF  $len = 0$  THEN 0
  ELSE  $Max(\{i \in 1 \dots len : boolOp(lhs, array[i])\})$ 

 $Insert(seq, prev, entry) \stackrel{\Delta}{=}$ 
  LET  $left \stackrel{\Delta}{=} \text{IF } prev = 0 \text{ THEN } \langle \rangle \text{ ELSE } SubSeq(seq, 1, prev)$ 
  LET  $right \stackrel{\Delta}{=} \text{IF } prev = Len(seq) \text{ THEN } \langle \rangle \text{ ELSE } SubSeq(seq, prev + 1, Len(seq))$  IN
   $Append(left, entry) \circ right$ 

 $AppendToSSFHistory(id, record) \stackrel{\Delta}{=}$ 
  LET  $ssfH \stackrel{\Delta}{=} ssfHistory[id]$ 
  LET  $step \stackrel{\Delta}{=} GetStepOffset(ssfH, record.pc, record.OpName, record.key)$  IN
  IF  $step > 0 \wedge ssfH[step] = record$  THEN UNCHANGED  $ssfHistory$ 
  ELSE  $ssfHistory' = [ssfHistory \text{ EXCEPT } ![id] = Append(@, record)]$ 

 $InsertReadToObjHistory(key, event) \stackrel{\Delta}{=}$ 
  LET  $objh \stackrel{\Delta}{=} objHistory[key]$  IN
  IF  $\exists i \in 1 \dots Len(objh) : objh[i].eid = event.eid$  THEN UNCHANGED  $objHistory$ 
  ELSE LET
     $vHistory \stackrel{\Delta}{=} [i \in 1 \dots Len(objh) \mapsto objh[i].version]$ 
     $prev \stackrel{\Delta}{=} Partition(vHistory, Len(objh), event.version, VersionHigherOrEq)$  IN
     $objHistory' = [objHistory \text{ EXCEPT } ![key] = Insert(objh, prev, event)]$ 

 $InsertWriteToObjHistory(key, event) \stackrel{\Delta}{=}$ 
  LET  $objh \stackrel{\Delta}{=} objHistory[key]$ 
  LET  $vHistory \stackrel{\Delta}{=} [i \in 1 \dots Len(objh) \mapsto objh[i].version]$ 
  LET  $prev \stackrel{\Delta}{=} Partition(vHistory, Len(objh), event.version, VersionHigher)$  IN
  IF  $prev < Len(objh) \wedge objh[prev + 1].eid = event.eid$  THEN UNCHANGED  $objHistory$ 
  ELSE  $objHistory' = [objHistory \text{ EXCEPT } ![key] = Insert(objh, prev, event)]$ 

 $TopoCandidates(edges) \stackrel{\Delta}{=}$ 
   $\{e1 \in edges :$ 
     $\forall e2 \in edges :$ 
     $e2[2] \neq e1[1]\}$ 

RECURSIVE  $TopoSort(-)$ 
 $TopoSort(edges) \stackrel{\Delta}{=}$ 
  IF  $edges = \{\}$  THEN TRUE

```

```

ELSE LET candidates  $\triangleq$  TopoCandidates(edges) IN
  IF candidates = {} THEN FALSE
  ELSE TopoSort(edges \ candidates)

GlobalDepsAcyclic(programDeps(-), dataDeps(-))  $\triangleq$ 
  LET pDeps  $\triangleq$  {edge  $\in$  EventIDs  $\times$  EventIDs :
     $\exists id \in SSFs : edge \in programDeps(id)$ }
    dDeps  $\triangleq$  {edge  $\in$  EventIDs  $\times$  EventIDs :
       $\exists key \in Objects : edge \in dataDeps(key)$ } IN
    TopoSort(pDeps  $\cup$  dDeps)

dataDeps(key)  $\triangleq$ 
  LET objh  $\triangleq$  objHistory[key] IN
  IF Len(objh)  $\leq$  1 THEN {}
  ELSE {<eid1, eid2>  $\in$  EventIDs  $\times$  EventIDs :
     $\exists i, j \in 1 \dots Len(objh) :$ 
       $i < j$ 
       $\wedge objh[i].eid = eid1$ 
       $\wedge objh[j].eid = eid2$ 
       $\wedge objh[i].opName = \text{"write"} \vee objh[j].opName = \text{"write"}$ }

|----- MODULE HMRead -----|

EXTENDS HMBase

objState is an ordered sequence of object versions
for the sake of reducing the state space, we reuse objHistory for objState

ObjVersionConsistent  $\triangleq$ 
   $\forall key \in Objects :$ 
    LET objh  $\triangleq$  objHistory[key] IN
     $\vee Len(objh) = 0$ 
     $\vee \exists id \in SSFs, pid \in 1 \dots NumPeers :$ 
      LET ssf  $\triangleq$  ssfState[id, pid]
      version  $\triangleq$  objh[Len(objh)].version IN
       $\wedge version.ts \leq ssf.cursorTS$ 
       $\wedge version.pc \leq ssf.pc$ 

sequential consistency
programDeps(id)  $\triangleq$ 
  LET ssfH  $\triangleq$  ssfHistory[id] IN
  IF Len(ssfH)  $\leq$  1 THEN {}

```

ELSE $\{\langle id, pc - 1 \rangle, \langle id, pc \rangle\} : pc \in 2 \dots Len(ssfh)\}$

Invariants $\triangleq$

$\wedge BaseInvariants$

$\wedge ObjVersionConsistent$

$\wedge GlobalDepsAcyclic(programDeps, dataDeps)$

Init $\triangleq BaseInit \wedge objState = [key \in Objects \mapsto 0]$

LookUpObjVersion(*objh*, *version*) $\triangleq$

LET *vHistory* $\triangleq [i \in 1 \dots Len(objh) \mapsto objh[i].version]$

prev $\triangleq Partition(vHistory, Len(objh), version, VersionHigher)$ IN

IF *prev* = 0 THEN [*ts* $\mapsto$ 0, *pc* $\mapsto$ 0]

ELSE *objh*[*prev*].*version*

Read(*id*, *pid*, *key*) $\triangleq$

LET

ssf $\triangleq ssfState[id, pid]$

cmd $\triangleq ssfCommand[id]$

pcNext $\triangleq ssf.pc + 1$

version $\triangleq LookUpObjVersion(objHistory[key],$

$[ts \mapsto ssf.cursorTS, pc \mapsto pcNext])$

IN

$\wedge pcNext \leq Len(cmd)$

$\wedge cmd[pcNext].opName = \text{“read”}$

$\wedge cmd[pcNext].key = key$

$\wedge ssf.nextOp = \text{“all”}$

$\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid].pc = pcNext]$

$\wedge \text{LET } record \triangleq [pc \mapsto pcNext,$

$opName \mapsto \text{“read”},$

$key \mapsto key,$

$ts \mapsto ssf.cursorTS,$

$version \mapsto version]$ IN

AppendToSSFHistory(*id*, *record*)

$\wedge \text{LET } event \triangleq [eid \mapsto \langle id, pcNext \rangle, version \mapsto version, opName \mapsto \text{“read”}]$ IN

InsertReadToObjHistory(*key*, *event*)

$\wedge \text{UNCHANGED } \langle globalSeqnum, ssfCommand, objState \rangle$

Write(*id*, *pid*, *key*) $\triangleq$

```

LET
   $ssf \triangleq ssfState[id, pid]$ 
   $cmd \triangleq ssfCommand[id]$ 
   $pcNext \triangleq ssf.pc + 1$ 
IN
   $\wedge pcNext \leq Len(cmd)$ 
   $\wedge cmd[pcNext].opName = \text{"write"}$ 
   $\wedge cmd[pcNext].key = key$ 
   $\wedge ssf.nextOp = \text{"all"}$ 
   $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"writelog"},$ 
 $!.pc = pcNext,$ 
 $!.tmp.key = key]]$ 

ssf and obj history is updated in the logging step

 $\wedge \text{UNCHANGED } \langle globalSeqnum, ssfCommand, ssfHistory, objState, objHistory \rangle$ 

GetOrLogWrite( $id, pid$ )  $\triangleq$ 
  LET
     $ssf \triangleq ssfState[id, pid]$ 
     $key \triangleq ssf.tmp.key$ 
     $version \triangleq [ts \mapsto globalSeqnum + 1, pc \mapsto ssf.pc]$ 
     $step \triangleq GetStepOffset(ssfHistory[id], ssf.pc, \text{"write"}, key)$ 
  IN
    IF  $step = 0$  THEN LET
       $wlog \triangleq [pc \mapsto ssf.pc,$ 
 $opName \mapsto \text{"write"},$ 
 $key \mapsto key,$ 
 $ts \mapsto globalSeqnum + 1,$ 
 $version \mapsto version]$  IN
       $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"all"},$ 
 $!.cursorTS = globalSeqnum + 1]]$ 
       $\wedge ssfHistory' = [ssfHistory \text{ EXCEPT } ![id] = Append(@, wlog)]$ 
       $\wedge globalSeqnum' = globalSeqnum + 1$ 
       $\wedge \text{LET } event \triangleq [eid \mapsto \langle id, ssf.pc \rangle, version \mapsto version, opName \mapsto \text{"write"}] \text{ IN}$ 
 $objHistory' = [objHistory \text{ EXCEPT } ![key] = Append(@, event)]$ 
    ELSE LET  $wlog \triangleq ssfHistory[id][step]$  IN
       $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"all"},$ 
 $!.cursorTS = wlog.ts]]$ 
       $\wedge \text{UNCHANGED } \langle globalSeqnum, ssfHistory, objHistory \rangle$ 

WriteLog( $id, pid$ )  $\triangleq$ 
   $\wedge ssfState[id, pid].nextOp = \text{"writelog"}$ 

```

$$\wedge \text{GetOrLogWrite}(id, pid)$$

$$\wedge \text{UNCHANGED } \langle objState, ssfCommand \rangle$$

$$Next \triangleq$$

$$\vee BaseNext$$

$$\vee \exists id \in SSFs, pid \in 1 \dots NumPeers : WriteLog(id, pid)$$

$$\vee \exists id \in SSFs, pid \in 1 \dots NumPeers, key \in Objects : \vee Read(id, pid, key)$$

$$\vee Write(id, pid, key)$$

$$Spec \triangleq Init \wedge \quad \square[Next] \langle globalSeqnum, ssfCommand,$$

$$ssfState, ssfHistory,$$

$$objState, objHistory \rangle$$

THEOREM $Spec \implies \square Invariants$

MODULE *HMWrite*

EXTENDS *HMBase*

$$ObjectStateTypeOK \triangleq$$

$$objState \in [Objects \rightarrow [version : Versions]]$$

$$ObjVersionConsistent \triangleq$$

$$\forall key \in Objects :$$

$$\text{LET } obj \triangleq objState[key] \text{ IN}$$

$$\exists id \in SSFs, pid \in 1 \dots NumPeers :$$

$$\text{LET } ssf \triangleq ssfState[id, pid] \text{ IN}$$

$$\wedge obj.version.ts \leq ssf.cursorTS$$

$$\wedge obj.version.pc \leq ssf.pc$$

sequential consistency except that consecutive writes may commute

$$programDeps(id) \triangleq$$

$$\text{LET } ssfh \triangleq ssfHistory[id]$$

$$eids \triangleq \{id\} \times (1 \dots MaxOps) \text{ IN}$$

$$\text{IF } Len(ssfh) \leq 1 \text{ THEN } \{\}$$

$$\text{ELSE } \{ \langle eid1, eid2 \rangle \in eids \times eids :$$

$$\quad \exists i, j \in 1 \dots Len(ssfh) :$$

$$\quad \wedge i < j$$

$$\quad \wedge ssfh[i].pc = eid1[2]$$

$$\quad \wedge ssfh[j].pc = eid2[2]$$

$$\quad \wedge ssfh[i].opName = \text{"read"} \vee ssfh[j].opName = \text{"read"} \}$$

$$Invariants \triangleq$$

$\wedge \text{BaseInvariants}$
 $\wedge \text{ObjVersionConsistent}$
 $\wedge \text{GlobalDepsAcyclic}(\text{programDeps}, \text{dataDeps})$

$\text{InitObjectState} \triangleq$
 $\text{objState} = [\text{key} \in \text{Objects} \mapsto [\text{version} \mapsto [\text{ts} \mapsto 0,$
 $\text{pc} \mapsto 0]]]$

$\text{Init} \triangleq \text{BaseInit} \wedge \text{InitObjectState}$

$\text{Read}(id, pid, key) \triangleq$
 LET
 $\text{cmd} \triangleq \text{ssfCommand}[id]$
 $\text{ssf} \triangleq \text{ssfState}[id, pid]$
 $\text{pcNext} \triangleq \text{ssf.pc} + 1$
 $\text{ts} \triangleq \text{ssf.cursorTS}$
 $\text{version} \triangleq \text{objState}[key].\text{version}$
 IN
 $\wedge \text{pcNext} \leq \text{Len}(\text{cmd})$
 $\wedge \text{cmd}[\text{pcNext}].\text{opName} = \text{"read"}$
 $\wedge \text{cmd}[\text{pcNext}].\text{key} = \text{key}$
 $\wedge \text{ssf.nextOp} = \text{"all"}$
 $\wedge \text{ssfState}' = [\text{ssfState} \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.\text{nextOp} = \text{"readlog"},$
 $\text{!.pc} = \text{pcNext},$
 $\text{!.tmp} = [\text{key} \mapsto \text{key}, \text{version} \mapsto \text{version}]]]$
 $\text{ssf and obj history is updated in the logging step}$
 $\wedge \text{UNCHANGED } \langle \text{globalSeqnum}, \text{ssfCommand}, \text{ssfHistory}, \text{objState}, \text{objHistory} \rangle$

$\text{GetOrLogRead}(id, pid) \triangleq$
 LET
 $\text{ssf} \triangleq \text{ssfState}[id, pid]$
 $\text{key} \triangleq \text{ssf.tmp.key}$
 $\text{version} \triangleq \text{ssf.tmp.version}$
 $\text{step} \triangleq \text{GetStepOffset}(\text{ssfHistory}[id], \text{ssf.pc}, \text{"read"}, \text{key})$
 IN
 $\text{IF step} = 0 \text{ THEN LET}$
 $\text{rlog} \triangleq [\text{pc} \mapsto \text{ssf.pc},$
 $\text{opName} \mapsto \text{"read"},$
 $\text{key} \mapsto \text{key},$
 $\text{ts} \mapsto \text{globalSeqnum} + 1,$

$$\begin{aligned}
& \text{version} \mapsto \text{version}] \text{ IN} \\
& \wedge \text{ssfState}' = [\text{ssfState} \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.\text{nextOp} = \text{"all"}, \\
& \hspace{20em} !.\text{cursorTS} = \text{globalSeqnum} + 1]] \\
& \wedge \text{ssfHistory}' = [\text{ssfHistory} \text{ EXCEPT } ![id] = \text{Append}(@, rlog)] \\
& \wedge \text{globalSeqnum}' = \text{globalSeqnum} + 1 \\
& \wedge \text{LET event} \stackrel{\Delta}{=} [eid \mapsto \langle id, \text{ssf.pc} \rangle, \text{version} \mapsto \text{version}, \text{opName} \mapsto \text{"read"}] \text{ IN} \\
& \quad \text{InsertReadToObjHistory}(\text{key}, \text{event}) \\
\text{ELSE} \quad & \text{LET rlog} \stackrel{\Delta}{=} \text{ssfHistory}[id][step] \text{ IN} \\
& \wedge \text{ssfState}' = [\text{ssfState} \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.\text{nextOp} = \text{"all"}, \\
& \hspace{20em} !.\text{cursorTS} = \text{rlog.ts}]] \\
& \wedge \text{UNCHANGED} \langle \text{globalSeqnum}, \text{ssfHistory}, \text{objHistory} \rangle \\
\\
\text{ReadLog}(id, pid) & \stackrel{\Delta}{=} \\
& \wedge \text{ssfState}[id, pid].\text{nextOp} = \text{"readlog"} \\
& \wedge \text{GetOrLogRead}(id, pid) \\
& \wedge \text{UNCHANGED} \langle \text{objState}, \text{ssfCommand} \rangle \\
\\
\text{UpdateObj}(id, pid, key) & \stackrel{\Delta}{=} \\
& \text{LET} \\
& \text{ssf} \stackrel{\Delta}{=} \text{ssfState}[id, pid] \\
& \text{pcNext} \stackrel{\Delta}{=} \text{ssf.pc} + 1 \\
& \text{version} \stackrel{\Delta}{=} [ts \mapsto \text{ssf.cursorTS}, pc \mapsto \text{pcNext}] \\
& \text{event} \stackrel{\Delta}{=} [eid \mapsto \langle id, \text{pcNext} \rangle, \text{version} \mapsto \text{version}, \text{opName} \mapsto \text{"write"}] \\
& \text{obj} \stackrel{\Delta}{=} \text{objState}[key] \\
& \text{IN} \\
& \text{IF } \text{VersionHigher}(\text{version}, \text{obj.version}) \text{ THEN} \\
& \quad \wedge \text{objState}' = [\text{objState} \text{ EXCEPT } ![key].\text{version} = \text{version}] \\
& \quad \wedge \text{objHistory}' = [\text{objHistory} \text{ EXCEPT } ![key] = \text{Append}(@, \text{event})] \\
& \text{ELSE IF } \text{VersionHigher}(\text{obj.version}, \text{version}) \text{ THEN} \\
& \quad \wedge \text{InsertWriteToObjHistory}(\text{key}, \text{event}) \\
& \quad \wedge \text{UNCHANGED } \text{objState} \\
& \text{ELSE UNCHANGED } \langle \text{objState}, \text{objHistory} \rangle \\
\\
\text{Write}(id, pid, key) & \stackrel{\Delta}{=} \\
& \text{LET} \\
& \text{cmd} \stackrel{\Delta}{=} \text{ssfCommand}[id] \\
& \text{ssf} \stackrel{\Delta}{=} \text{ssfState}[id, pid] \\
& \text{pcNext} \stackrel{\Delta}{=} \text{ssf.pc} + 1 \\
& \text{version} \stackrel{\Delta}{=} [ts \mapsto \text{ssf.cursorTS}, pc \mapsto \text{pcNext}] \\
& \text{record} \stackrel{\Delta}{=} [pc \mapsto \text{pcNext}, \\
& \quad \text{opName} \mapsto \text{"write"}, \\
& \quad \text{key} \mapsto \text{key},
\end{aligned}$$

$$\begin{aligned}
& ts \mapsto ssf.cursorTS, \\
& version \mapsto version] \\
\text{IN} \\
& \wedge pcNext \leq Len(cmd) \\
& \wedge cmd[pcNext].opName = \text{“write”} \\
& \wedge cmd[pcNext].key = key \\
& \wedge ssf.nextOp = \text{“all”} \\
& \wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid].pc = pcNext] \\
& \wedge AppendToSSFHistory(id, record) \\
& \wedge UpdateObj(id, pid, key) \\
& \wedge \text{UNCHANGED } \langle globalSeqnum, ssfCommand \rangle \\
Next & \triangleq \\
& \vee BaseNext \\
& \vee \exists id \in SSFs, pid \in 1 \dots NumPeers : ReadLog(id, pid) \\
& \vee \exists id \in SSFs, pid \in 1 \dots NumPeers, key \in Objects : \vee Read(id, pid, key) \\
& \hspace{15em} \vee Write(id, pid, key) \\
Spec & \triangleq Init \wedge \quad \Box[Next] \langle globalSeqnum, ssfCommand, \\
& \hspace{10em} ssfState, ssfHistory, \\
& \hspace{10em} objState, objHistory \rangle
\end{aligned}$$

THEOREM $Spec \implies \Box Invariants$

MODULE *HMWriteExt*

EXTENDS *HMWrite*

$$\begin{aligned}
ReadTSs & \triangleq [ts : Nat, version : Versions] \\
ReadTSHigher(left, right) & \triangleq \\
& \vee VersionHigher(left.version, right.version) \\
& \vee left.version = right.version \wedge left.ts > right.ts \\
ExtObjectStateTypeOK & \triangleq \\
& objState \in [Objects \rightarrow [readTS : Seq(ReadTSs), \\
& \hspace{10em} version : Versions]] \\
ExtObjVersionConsistent & \triangleq \\
& \wedge ObjVersionConsistent \\
& \wedge \forall key \in Objects : \\
& \quad \text{LET } obj \triangleq objState[key] \text{ IN} \\
& \quad \forall i \in 1 \dots Len(obj.readTS) :
\end{aligned}$$

$$\begin{aligned}
& ExtObjReadTSMonotone \stackrel{\Delta}{=} \\
& \quad \forall key \in Objects : \\
& \quad \quad LET readTS \stackrel{\Delta}{=} objState[key].readTS \text{ IN} \\
& \quad \quad \forall i, j \in 1 \dots Len(readTS) : \\
& \quad \quad \quad i < j \implies \wedge VersionHigherOrEq(readTS[j].version, readTS[i].version) \\
& \quad \quad \quad \wedge ReadTSHigher(readTS[j], readTS[i])
\end{aligned}$$
$$\begin{aligned} \text{ExtInvariants} &\triangleq \\ &\wedge \text{BaseInvariants} \\ &\wedge \text{ExtObjectTypeOK} \\ &\wedge \text{ExtObjVersionConsistent} \\ &\wedge \text{ExtObjReadTSMonotone} \\ &\wedge \text{GlobalDepsAcyclic}(\text{extProgramDeps}, \text{dataDeps}) \end{aligned}$$
$$ExtInit \stackrel{\Delta}{=} BaseInit \wedge ExtInitObjectState$$

55

$$\begin{aligned}
& obj \triangleq objState[key] \\
& version \triangleq obj.version \\
& readTS \triangleq obj.readTS \\
& \text{IN} \\
& \wedge pcNext \leq Len(cmd) \\
& \wedge cmd[pcNext].opName = \text{“read”} \\
& \wedge cmd[pcNext].key = key \\
& \wedge ssf.nextOp = \text{“all”} \\
& \wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{“readlog”,} \\
& \hspace{15em} !.pc = pcNext, \\
& \hspace{15em} !.tmp = [key \mapsto key, version \mapsto version]]] \\
& \wedge \text{IF } \wedge ts > version.ts \\
& \quad \text{the second condition is just an optimization} \\
& \quad \wedge Len(readTS) = 0 \vee ts > readTS[Len(readTS)].ts \text{ THEN LET} \\
& \quad \quad myReadTS \triangleq [ts \mapsto ssf.cursorTS, version \mapsto version] \\
& \quad \quad prev \triangleq Partition(readTS, Len(readTS), myReadTS, ReadTSHigher) \text{ IN} \\
& \quad \quad \quad objState' = [objState \text{ EXCEPT } ![key].readTS = Insert(readTS, prev, myReadTS)] \\
& \quad \text{ELSE UNCHANGED } objState \\
& \quad \text{ssf and obj history is updated in the logging step} \\
& \quad \text{UNCHANGED } \langle globalSeqnum, ssfCommand, ssfHistory, objHistory \rangle \\
& CheckTSInversion(readTS, version) \triangleq \\
& \quad \text{LET} \\
& \quad \quad versions \triangleq [i \in 1 \dots Len(readTS) \mapsto readTS[i].version] \\
& \quad \quad prev \triangleq Partition(versions, Len(readTS), version, VersionHigher) \text{ IN} \\
& \quad \quad \quad prev \neq 0 \wedge readTS[prev].ts > version.ts \\
& ExtWrite(id, pid, key) \triangleq \\
& \quad \text{LET} \\
& \quad \quad cmd \triangleq ssfCommand[id] \\
& \quad \quad ssf \triangleq ssfState[id, pid] \\
& \quad \quad pcNext \triangleq ssf.pc + 1 \\
& \quad \quad version \triangleq [ts \mapsto ssf.cursorTS, pc \mapsto pcNext] \\
& \quad \quad record \triangleq [pc \mapsto pcNext, \\
& \quad \quad \quad opName \mapsto \text{“write”,} \\
& \quad \quad \quad key \mapsto key, \\
& \quad \quad \quad ts \mapsto ssf.cursorTS, \\
& \quad \quad \quad version \mapsto version] \\
& \quad \text{IN} \\
& \quad \wedge pcNext \leq Len(cmd) \\
& \quad \wedge cmd[pcNext].opName = \text{“write”}
\end{aligned}$$

```

 $\wedge cmd[pcNext].key = key$ 
 $\wedge ssf.nextOp = \text{"all"}$ 
 $\wedge \text{IF } CheckTSInversion(objState[key].readTS, version) \text{ THEN}$ 
     $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"writelog"},$ 
 $\quad \quad \quad !.pc = pcNext,$ 
 $\quad \quad \quad !.tmp = [key \mapsto key, version \mapsto version]]]$ 
 $\wedge \text{UNCHANGED } ssfHistory$ 
 $\text{ELSE}$ 
     $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid].pc = pcNext]$ 
     $\wedge AppendToSSFHistory(id, record)$ 
 $\wedge UpdateObj(id, pid, key)$ 
 $\wedge \text{UNCHANGED } \langle globalSeqnum, ssfCommand \rangle$ 

 $GetOrLogWrite(id, pid) \triangleq$ 
     $\text{LET}$ 
     $ssf \triangleq ssfState[id, pid]$ 
     $key \triangleq ssf.tmp.key$ 
     $version \triangleq ssf.tmp.version$ 
     $step \triangleq GetStepOffset(ssfHistory[id], ssf.pc, \text{"write"}, key)$ 
     $\text{IN}$ 
     $\text{IF } step = 0 \text{ THEN LET}$ 
         $wlog \triangleq [pc \mapsto ssf.pc,$ 
         $\quad opName \mapsto \text{"write"},$ 
         $\quad key \mapsto key,$ 
         $\quad ts \mapsto globalSeqnum + 1,$ 
         $\quad version \mapsto version] \text{ IN}$ 
         $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"all"},$ 
 $\quad \quad \quad !.cursorTS = globalSeqnum + 1]]$ 
         $\wedge ssfHistory' = [ssfHistory \text{ EXCEPT } ![id] = Append(@, wlog)]$ 
         $\wedge globalSeqnum' = globalSeqnum + 1$ 
     $\text{ELSE LET } wlog \triangleq ssfHistory[id][step] \text{ IN}$ 
         $\wedge ssfState' = [ssfState \text{ EXCEPT } ![id, pid] = [@ \text{ EXCEPT } !.nextOp = \text{"all"},$ 
 $\quad \quad \quad !.cursorTS = wlog.ts]]$ 
         $\wedge \text{UNCHANGED } \langle ssfHistory, globalSeqnum \rangle$ 

 $WriteLog(id, pid) \triangleq$ 
     $\wedge ssfState[id, pid].nextOp = \text{"writelog"}$ 
     $\wedge GetOrLogWrite(id, pid)$ 
     $\wedge \text{UNCHANGED } \langle objState, objHistory, ssfCommand \rangle$ 

 $ExtNext \triangleq$ 
     $\vee BaseNext$ 

```

$$\begin{aligned}
& \vee \exists id \in SSFs, pid \in 1 \dots NumPeers : \vee ReadLog(id, pid) \\
& \qquad \qquad \qquad \vee WriteLog(id, pid) \\
& \vee \exists id \in SSFs, pid \in 1 \dots NumPeers, key \in Objects : \vee ExtRead(id, pid, key) \\
& \qquad \qquad \qquad \vee ExtWrite(id, pid, key) \\
\\
& ExtSpec \triangleq ExtInit \wedge \Box[ExtNext]\langle globalSeqnum, ssfCommand, \\
& \qquad \qquad \qquad ssfState, ssfHistory, \\
& \qquad \qquad \qquad objState, objHistory \rangle
\end{aligned}$$

THEOREM $ExtSpec \implies \Box ExtInvariants$